

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (ПГУ)**

Горюнов Ю.Ю., Леонова Т.Ю.

Объектно-ориентированные языки и системы программирования

Учебное пособие

**Пенза
Издательство ПГУ
2025**

Введение. Объектно-ориентированные языки и системы программирования

Под *объектно-ориентированным языком* понимают язык программирования, который позволяет создавать программы на основе классов, объединяющих в себе переменные (поля) и функции (методы), и поддерживает ключевые принципы объектно-ориентированного программирования (ООП):

- *инкапсуляцию* – объединение полей и методов в одном классе и возможности их сокрытия от вызова извне класса;
- *наследование* – возможность создавать новые классы (классы-наследники) на основе существующих;
- *полиморфизм* – каждый класс-потомок может перекрывать методы родителя, то есть у родителя и потомка могут одноимённые методы, имеющие разные тела или список аргументов.

Под объектно-ориентированными системами программирования (ООСП) понимаются языки программирования и инструментарии, поддерживающие технологию ООП.

ООСП пользуются огромной популярностью в мире. Это обусловлено тем, что они позволяют наиболее адекватно моделировать сложные информационные процессы.

К числу ООСП относятся следующие системы: Delphi и C++ Borland (фирмы Borland); Visual Basic, Visual C++ Microsoft Visual Studio 2019/2022 (фирмы Microsoft), которую будем использовать для выполнения лабораторных работ.

Глава 1. Объектно-ориентированные языки программирования

1.1. Стили программирования, основные принципы, история развития

В настоящее время наиболее популярны два вида (две парадигмы¹) программирования: структурное и ООП.

В структурном программировании:

- данные (переменные, структуры) пассивны и обрабатываются функциями;
- программа представляет собой последовательность операторов, реализующих команды алгоритмов для обработки данных, говоря иначе, программа – это функция, которая принимает входные данные и возвращает результат.

В объектно-ориентированном программировании:

- данные (поля или свойства) и функции (методы) объединены в классы;
- программа представляет собой совокупность обменивающихся сообщениями классов, посредством которых моделируются сущности предметной области.

¹ В переводе с греческого парадигма означает пример, образец.

Эволюция языков и стилей

Эволюция шла в сторону повышения уровня абстракции и улучшения управления сложностью:

- машинные коды;
- машинно-ориентированный язык ассемблера;
- языки, поддерживающие структурное программирование: Паскаль, С;
- языки, поддерживающие ООП: С++, Java, С#;
- языки, поддерживающие функциональное программирование: F, Python, JavaScript, Scala.

Основной «движущей силой» эволюции является борьба со сложностью больших проектов.

Объектно-ориентированные языки программирования

- «Чистые» ОО-языки: Smalltalk, Ruby.
- Гибридные языки (поддерживают ООП и другие парадигмы):
 - С++: надстройка ООП над С (С с классами);
 - Java, С#: созданы с ориентацией на ООП, но сохраняют процедурные элементы;
 - Python, JavaScript: мультипарадигмальные языки, где ООП — одна из основных, но не единственная модель².

1.2. Тип класс, экземпляры класса

Класс – это тип данных, состоящий из объявлений переменных (полей класса) и функций (методов класса).

Переменные типа класс называются *экземплярами (объектами) класса*.

Свойства классов

- *Инкапсуляция* — соединение объявлений переменных и функций в одном классе плюс возможность запрета использования полей и/или методов вне класса.
- *Наследование* — каждый класс может иметь произвольное количество классов-наследников, каждый из которых автоматически наследует все поля и методы родителя и может объявлять свои поля и методы.
- *Полиморфизм* — каждый класс-потомок может перекрывать методы родителя, то есть у родителя и потомка могут одноимённые методы, имеющие разные тела или список аргументов.

² Модель – это упрощённое представление реальности, созданное для решения конкретной задачи. Она выделяет только те детали, которые важны в данном контексте, и игнорирует всё остальное.

Объявление классов (упрощенно)

```
class имя_класса
{
    private:
        // объявления скрытых полей и методов класса
    public:
        // объявления открытых (общедоступных) полей и методов класса
};
```

где private и public – спецификаторы доступа:

- действие спецификатора распространяется до следующего спецификатора или до конца объявления класса «};»,
- число и порядок следования спецификаторов не оговаривается, по умолчанию действует спецификатор private.

- ✓ Классы могут быть глобальными (объявлены вне блока), либо локальными (объявлены внутри блока, например, внутри функции).
- ✓ Поля класса могут иметь любой тип, кроме типа этого же класса.
- ✓ Методы класса можно объявлять внутри класса, либо внутри класса объявляется прототип метода, а вне класса – полное объявление.

Пример объявления класса

```
class A
{
    int N;           // скрытое поле.
    public:
    void inpN(void) // открытый метод - объявлен внутри класса.
        {cout<<"N="; cin>>N;}
    void outN(void); // прототип метода открытого метода.
};
void A::outN(void) // внешнее объявление метода.
    {cout<<"\n"<< N;}
```

Объявление экземпляров класса

- ✓ типа класс:
имя_класса имя_экземпляра [, имя_экземпляра, ...];
- ✓ типа указатель на класс:
имя_класса *имя_экземпляра [, *имя_экземпляра, ...];

Примечание. Для каждого из объявленных экземпляров типа указатель на класс следует выделить память оператором new.

Доступ к полям и методам экземпляров

Если экземпляр объявлен типа класс, то для доступа к его (открытым) полям и методам используется операция точка ".", а если типа указатель на класс, то операция стрелка "->".

Пример объявления экземпляров и доступа к их полям и методам

```
A a, *b;  
b = new A; // выделение памяти для экземпляра b.  
int K=a.N; // ошибка – поле N объявлено закрытым.  
a.inpN( );  
b -> outN( );
```

1.3. Конструкторы и деструкторы классов

Конструкторами и деструкторами называются методы классов, которые обладают особыми свойствами.

Свойства конструкторов

- ✓ Имена конструкторов совпадают с именами классов.
- ✓ Конструкторы должны быть общедоступными и не возвращать никакого значения, даже типа **void**.
- ✓ Если конструктор в классе не объявлен, то компилятор автоматически создаёт один конструктор.
- ✓ В классе может быть несколько конструкторов и все с различными списками аргументов.
- ✓ Конструктор, который может быть вызван без параметров, называется конструктором по умолчанию.
- ✓ Конструкторы вызываются при объявлении экземпляров классов.
- ✓ Конструкторы обычно объявляются для инициализации полей класса.

Примеры объявления конструкторов

- Объявление одного конструктора по умолчанию.

<pre>class A { int M, N; public: A (int m = 0, int n = 5) {M = m; N = n;} };</pre>	<u>объявление экземпляров класса</u> A a; // по умолчанию: a.M=0; a.N=5. A b(10, 15); // b.M = 10, b.N = 15; A *d = new A (3, 4); // d-> M=3, d-> N=4
--	--

- Объявление двух конструкторов с параметрами.

<pre>class A { public: A(int N) { cout<< N+1<<endl;} A(double X) { cout<< 2*X<<endl;} };</pre>	<u>ВЫЗОВ КОНСТРУКТОРОВ</u> A a1(5); // в консоли 6. A a2(0.7); // в консоли 1,4.
--	--

Деструкторы классов

- ✓ Имя деструктора начинается с символа “~” (тильда), за которым следует имя класса.
- ✓ Деструктор не может иметь параметров и возвращаемого значения.

- ✓ Если деструктор не объявлен, то компилятор создаёт его автоматически.
- ✓ Деструктор вызывается автоматически при выходе экземпляра из области видимости:
 - для локальных экземпляров при выходе из блока;
 - для глобальных экземпляров – при завершении программы.
- ✓ Деструктор необходимо объявлять явно в том случае, если в классе динамически выделяется память, например, для массивов (операцией new);
- ✓ Явно вызывать деструктор можно, но не рекомендуется.

Пример объявления деструктора.

```
Ar() {delete array[ ];};
```

Лабораторная работа № 1.1. Классы, экземпляры классов

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для табулирования функции Вашего варианта на отрезке $[a, b]$ с шагом h .

Требования к приложению

Следует объявить класс:

- закрытые поля класса a , b и h , значения которых инициализируются в конструкторе класса;
- открытые методы класса:
 - вычисление значения функции $f(n)$ Вашего варианта;
 - табулирование функции $f(n)$ Вашего варианта.

Содержание отчета

1. Формулировка задания Вашего варианта.
2. Теоретическая часть
 - понятие табулирования функции,
 - псевдокод вычисления функции $f(n)$ Вашего варианта.
3. Расчетная часть
 - результаты выполнения задания для двух экземпляров объявленного класса, первый из которых объявлен типа класс (поля инициализируются по умолчанию), а второй - типа указатель на класс (поля инициализируются явно);
 - результаты тестирования объявленного метода для вычисления $f(n)$;
 - текст консольного приложения с необходимыми комментариями.
4. Заключение.
5. Список использованной Вами литературы.

Варианты заданий

1. $f(n) = \tau(n)$ – число всех различных делителей числа n .
2. $f(n) = \varphi(n)$ – функция Эйлера.
3. $f(n) = \nu(n)$ – число всех различных простых делителей числа n .
4. $f(n) = \Omega(n)$ – число всех простых делителей числа n с учетом их кратности³.
5. $f(n) = \pi(n)$ – число простых чисел, не превосходящих n .

³ Если $n = p_1^{d_1} p_2^{d_2} \dots p_k^{d_k}$, то $\Omega(n) = d_1 + d_2 + \dots + d_k$.

6. $f(n) = \mu(n) = \begin{cases} 1, & \text{если } n = 1, \\ 0, & \text{если } n \text{ делится на } d^2, d > 1, \\ (-1)^{v(n)}, & \text{в остальных случаях,} \end{cases} \quad - \text{ функция Мёбиуса.}$
7. $f(n) = \sum_{d|n} \varphi(d)$, где суммирование ведется по всем делителям n ,
 $\varphi(d)$ - функция Эйлера.
8. $f(n) = \sum_{d|n} \mu(d)$, где суммирование ведется по всем делителям n , где
 $\mu(d)$ - функция Мёбиуса.
9. $f(n) = \mu(\varphi(n))$, где $\mu(d)$ - функция Мёбиуса, $\varphi(d)$ - функция Эйлера.
10. $f(n) = \sum_{k \leq n} \mu(k) \tau(k-1)$, где суммирование ведется по натуральным k ,
11. $\tau(d)$ – число всех различных делителей числа d , $\tau(0) = 0$.
12. $f(n) = \sum_{d|n} \mu^2(d)$, где суммирование ведется по всем делителям n ,
 $\mu(d)$ - функция Мёбиуса.
13. $f(n) = \mu^2(\varphi^3(n))$, где $\mu(d)$ - функция Мёбиуса, $\varphi(d)$ - функция Эйлера.
14. $f(n) = \frac{\ln x}{x} \cdot \pi(x)$, где $\pi(x)$ – число всех простых чисел, не превосходящих x .
15. $f(n) = \Omega(n) \cdot \Omega(n-1)$, где $\Omega(n)$ - число всех простых делителей числа n с учетом их кратности.
16. $f(n) = v(n) \cdot v(n+1)$, где $v(n)$ – число всех различных простых делителей числа n .

Рекомендации по выполнению задания

Задание. Создать в MS Visual Studio 2019/2022 на языке C++ консольное приложение для табулирования функции Вашего варианта на отрезке $[a, b]$ с шагом h .

Вариант 0. $f(n) = n^2 + n - 1$.

Выполнение задания

Теоретическая часть

Псевдокод вычисления функции $f(n) = n^2 + n - 1$.

Функция $f(n)$:

Вход: целое n ,

Выход: целое $n^2 + n - 1$

return $n^2 + n - 1$

Расчетная часть

Создать консольное приложение в MS VS (см. Приложение Б стр. 64)

Результаты выполнения задания

Табулирование функции $f(n)$

tF - экземпляр типа класс tabFun tF(7,15, 2);	ptF - экземпляр типа указатель на класс tabFun *ptF; ptF = new tabFun(10, 34, 5);
f(7) =55, f(9) =89, f(11) =131 f(13) =181, f(15) =239	f(10) =109, f(15) =239, f(20) =419, f(25) =649, f(30) =929

Тестирование функции $f(n) = n^2 + n - 1$

$f(0) = -1, f(1) = 1, f(-1) = -1, f(2) = 5$

Результат тестирования: правильный.

Текст консольного приложения с необходимыми комментариями

```
#include <iostream>
```

```
#include <Windows.h>
```

```
using namespace std;
```

```
class tabFun
```

```
{
```

```
    int a, b, h; // скрытые поля.
```

```
public:
```

```
    int f(int n) // открытый метод.
```

```
{
```

```
        return n * n + n - 1;
```

```
}
```

```
    tabFun(int A = 0, int B = 4, int H = 1) // конструктор по умолчанию.
```

```
        // инициализация скрытых полей:
```

```
{
```

```
        a = A; b = B; h = H;
```

```
}
```

```
    void tabular(void) // открытый метод для табулирования функции.
```

```
{
```

```
        for (int n = a; n <= b; n += h)
```

```
            cout << "\n f(" << n << ") =" << f(n);
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    SetConsoleCP(1251); // установка кодировки 1251 для ввода в консоль,
```

```
    SetConsoleOutputCP(1251); // установка кодировки 1251,
```

```
    tabFun tF(7, 15, 2); // объявление экземпляра типа класс,
```

```
    cout << "Тестирование f(n)";
```

```
    cout << "\nf(0)=" << tF.f(0) << "\n f(1)=" << tF.f(1);
```

```
    cout << "\nf(-1)=" << tF.f(-1) << "\n f(2)=" << tF.f(2);
```

```
    cout << "\n tF – экземпляр типа класс";
```

```

tF.tabular(); // вызов метода,
cout << "\n\n ptF – экземпляр типа указатель на класс";
tabFun* ptF; // объявление экземпляра типа указатель на класс,
ptF = new tabFun(10, 34, 5); // выделение памяти объявленному
экземпляру,
ptF->tabular(); // вызов метода.
cout << "\n\n"; system("pause");
return 1;
}

```

1.4. Наследование классов

Объявление класса-наследника

```

class имя_класса: [ключ доступа] имя_родителя [ , имя_родителя2... ]
{тело класса};

```

ключ доступа к родительскому классу: public, protected, private.

Пример объявления наследника

```

class A {...}; class B {...}; // классы родители.
class D : public A // D есть наследник A с ключом доступа public к родителю,
{...};
class E : protected A, public B // у класса E два родителя A и B:
{...}; // protected – ключ доступа к родителю A,
//public – ключ доступа к родителю B.

```

Следует отметить, что:

- ✓ наследуемые поля и методы **не перемещаются** в класс-потомок, а остаются в родительском классе.
- ✓ компилятор при обращении к полю или методу экземпляра пытается найти его объявление в классе-потомке, а если это не удаётся, то продолжает поиск в классе-родителе.

Класс-потомок, наследуя поля и методы всех классов-родителей, получает к ним доступ в соответствии с таблицей 1.

ключ доступа к родителю	спецификатор поля/метода в родителе	доступ в классе-потомке
private	private protected public	нет – через открытые методы родителя private – напрямую protected – напрямую public – напрямую
protected	private protected public	нет – через открытые методы родителя protected – напрямую protected – напрямую public – напрямую
public	private protected public	нет – через открытые методы родителя protected – напрямую protected – напрямую public – напрямую

Таблица 1

Особенности наследования классов

- ✓ Конструкторы и деструкторы не наследуются.
- ✓ Если в конструкторе потомка нет явного вызова конструктора родителя, то вызывается конструктор по умолчанию родителя.
- ✓ В случае нескольких классов родителей их конструкторы вызываются в порядке объявления.
- ✓ Если конструктор с параметрами класса-наследника должен вызвать конструктор с параметрами класса-родителя, то он должен быть явно вызван в конструкторе класса-потомка в виде:
 - имя_класса_потомка(char C) : имя_класса_родителя(C)
 {тело конструктора потомка}
 или (вызываем конструктор родителя по умолчанию)
 - имя_класса_потомка(int n = 5) : имя_класса_родителя(n) {}
- ✓ Если деструктор потомка не объявлен, он создаётся по умолчанию и вызывает деструкторы всех классов-предков.
- ✓ В деструкторе потомка обращение к деструктору родителя происходит автоматически.

Простое и множественное наследование

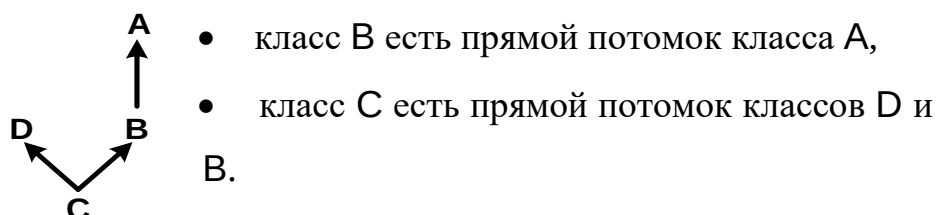
Класс называется *прямым родителем*, если он входит в список классов-предков при объявлении потомка.

Простым называется наследование, при котором потомок имеет одного прямого родителя, и множественным, если – несколько.

Если класс А есть прямой родитель класса В, а В есть прямой родитель класса С, то А называется косвенным родителем класса С, а С- косвенным потомком класса А.

Иерархию классов при наследовании часто изображают в виде орграфов, в которых дуга “→” читается “есть потомок”.

Пример иерархии классов



Важно: *стрелки направлены от потомков к родителям*, так как именно в таком порядке их объявления рассматривает компилятор.

1.4.1. Полиморфизм и виртуальные методы

Если экземпляр потомка объявлен **типа класс**, то при обращении к перекрытому в потомке методу родителя, компилятор сразу к нему и обратится.

Объявление экземпляра типа указатель на класс позволяет присваивать ему адрес не только родителя, но и всех его потомков, то есть, **посредством одной переменной** обращаться к экземпляру-родителю и всем экземплярам его потомков как прямым, так и косвенным.

Для того, чтобы компилятор вызвал перекрытый в потомке метод того класса, на который указывает указатель, необходимо родительский метод объявить *виртуальным*:

virtual тип_результата имя_метода (аргументы) {тело метода}

Пример. Пусть объявлена иерархия классов Father ← Son ← GrandSon:

Father	Son	GrandSon
<pre>class Father {public: int age; Father (int a = 55) {age = a;} virtual void who (void) {cout << "Отец";} };</pre>	<pre>class Son : public Father {public: int age; Son (int a = 35) {age = a;} void who (void) {cout << "Сын";} };</pre>	<pre>class GrandSon : public Son {public: int age; GrandSon (int a = 7) {age = a;} void who (void) {cout << "Внук";} };</pre>

В потомках классах Father этой иерархии перекрыт родительский метод who.

Объявим экземпляр типа указатель на класс-родитель и присвоим ему адреса экземпляров наследников:

Father *ptrF= new Father; Son S; GrandSon Gs;		
ptrF->who(); // на экране: Отец	ptrF = &S; ptrF->who(); // на экране: Сын	ptrF = &Gs; ptrF->who(); // на экране: Внук

Особенности виртуальных методов

1. Если в родительском классе метод объявлен виртуальным, то метод в потомке с тем же именем и набором аргументов автоматически становится виртуальным, а с отличающимся набором аргументов – обычным.
2. Виртуальные методы наследуются, то есть в потомке они перекрываются только для задания другого тела.
3. Конструкторы и деструкторы не могут быть виртуальными.

Лабораторная работа № 1.2. Конструкторы, деструкторы, наследование, полиморфизм

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для решения задачи Вашего варианта.

Требования к приложению

- 1) объявить класс-родитель для:
 - объявления двумерного $m \times n$ массива Ar;
 - инициализации массива Ar случайными вещественными числами из отрезка $[A; B]$;
 - вывода элементов массива на экран;

- 2) объявить класс-потомок с методом для решения задачи Вашего варианта над массивом, который является полем родительского класса;
- 3) проконтролировать вызовы конструкторов и деструкторов.

Содержание отчета

1. Формулировка задания, требования к приложению.
2. Расчётная часть
 - результат выполнения задания;
 - диаграмма классов (создание диаграмм классов см. стр. 60);
 - текст приложения с необходимыми комментариями;
3. Заключение.
4. Список использованной Вами литературы.

Варианты заданий

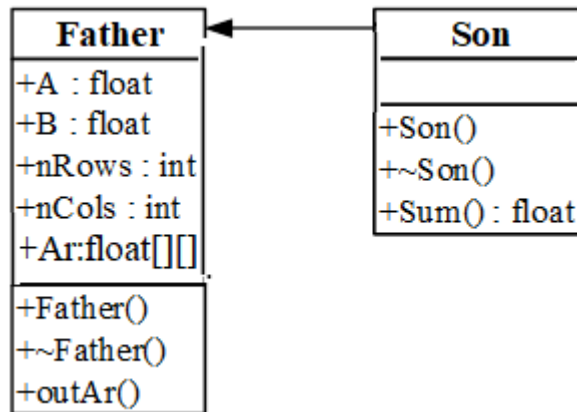
Во всех вариантах массивы двумерные, с вещественными элементами.

1. Отсортировать каждую строку двумерного массива по не убыванию.
2. "Перевернуть" каждый столбец двумерного массива.
3. "Перевернуть" каждую строку двумерного массива.
4. Сколько в двумерном массиве элементов с четной положительной целой частью.
5. Найти наибольший среди элементов двумерного массива с четной суммой индексов.
6. Найти наименьший элемент двумерного массива среди элементов, сумма индексов которых есть простое число.
7. Сколько в двумерном массиве элементов с нечетной отрицательной целой частью.
8. Сколько в двумерном массиве элементов с нечетной положительной целой частью.
9. Заменить в двумерном массиве отрицательные элементы нулями.
10. Заменить в двумерном массиве положительные элементы единицами.
11. Вывести на экран все элементы двумерного массива, которые больше своего «соседа» слева и меньше своего «соседа» справа в «своей» строке.
12. Вывести на экран все элементы двумерного массива, которые больше своего «соседа» сверху и меньше своего «соседа» снизу в «своем» столбце.
13. Найти сумму элементов двумерного массива, первые индексы которых есть простые числа, а вторые – составные.
14. Найти произведение элементов двумерного массива, оба индекса которых есть составные числа.
15. Сколько в двумерном массиве элементов, целая часть которых представима в виде суммы двух простых чисел.

Рекомендации по выполнению задания

Вариант 0. Вычислить сумму элементов массива

Диаграмма классов



Текст консольного приложения с необходимыми комментариями

```
#include <iostream>
#include <Windows.h>
#include <stdlib.h> // для функций srand и rand,
#include <ctime> // для функции time().
using namespace std;
class Father // Класс-родитель.
{ public:
    float A, B; // концы отрезка.
    int nRows, nCols; // число строк и столбцов.
    float** Ar; // динамический двумерный массив.

    // Конструктор по умолчанию:
    Father(int n = 5, int m = 7, float a = -20.0, float b = 20.0)
    { cout << "\n Вызван конструктор класса-родителя Father";
      nRows = n; nCols = m; A = a; B = b;

      // Объявление двумерного динамического массива:
      Ar = new float* [nRows];
      for (int k = 0; k < nRows; k++) Ar[k] = new float[nCols];
      // Установка начального случайного числа:
      srand(time(0));
      // Инициализация двумерного динамического массива:
      for (int i = 0; i < nRows; i++)
          for (int k = 0; k < nCols; k++)
              { float t = (float)(rand() % 100) / 100;
                Ar[i][k] = (b - a) * t + a;
              }
    }

    ~Father() // Деструктор.
    { cout << "\n Деструктор класса-родителя Father";
      delete [ ] Ar; // Освобождаем память.
```

```

    }
    void outAr() // Вывод элементов массива:
    { cout << "\n Массив Ar \n";
      for (int i = 0; i < nRows; i++)
        { for (int k = 0; k < nCols; k++) cout << Ar[i][k] << " ";
          cout << "\n";
        }
    }
}; // завершение объявления класса Father.
class Son : public Father
{ public:
  // В конструкторе класса-наследника явно обращаемся
  // к конструктору класса-родителя:
  Son(int n = 5, int m = 7, float a = -20.0, float b = 20.0) : Father(n, m, a, b)
  { cout << "\n Конструктор класса-наследника Son";
  }
  ~Son()
  {cout << "\n Деструктор класса-наследника Son";
  }
  float Sum() // Сумма элементов массива Ar – решение задачи Варианта 0.
  {float s = 0;
    for (int i = 0; i < nRows; i++)
      for (int k = 0; k < nCols; k++)
        s += Ar[i][k];
    return s;
  }
}; // завершение объявления класса Son.
int main()
{ SetConsoleCP(1251); // установка кодировки 1251 для ввода в консоль,
  SetConsoleOutputCP(1251); // установка кодировки 1251,
  // Для контроля вызовов конструкторов и деструкторов
  // экземпляр класса-наследника объявляем в блоке:
  { // объявление наследника;
    Son s(3, 2);
    // значение наследованных полей: nRows = 3, nCols = 2, A = -20, B =20.
    s.outAr(); // вывод элементов массива Ar;
    cout << "\n Сумма = " << s.Sum(); // сумма элементов массива.
  }
  cout << "\n\n"; system("pause");
  return 1;
}

```

1.5. Шаблоны классов

Шаблон класса – это класс с параметрами, значениями которых являются типы данных.

Объявление шаблона класса

```
template <параметры шаблона класса>  
    объявление_класса
```

- ✓ Параметры шаблона перечисляются через запятую.
- ✓ В качестве параметров могут использоваться обозначения как стандартных типов, так и типов пользователя
- ✓ Внутри шаблона класса параметры могут использоваться везде, где допустима спецификация типа.

Пример объявления шаблона класса.

```
template <class t> class A  
{ t N; // N - закрытое поле типа t,  
  public:  
    A (t n = 0)  
      {N = n;} // конструктор по умолчанию.  
  // ...  
};
```

Объявление экземпляра шаблона класса

```
имя_шаблона <значения параметров шаблона>  
    имя_экземпляра | (значения аргументов конструктора по умолчанию) | ;
```

Примеры объявления экземпляров шаблонов класса

```
A <int> a;           // вызывается конструктор по умолчанию,  
A <double> b(1.15);
```

Встретив объявление экземпляра шаблона класса, компилятор выполняет два действия:

- 1) заменяет в объявлении шаблона параметры-типы на указанные в угловых скобках типы и получает объявление класса;
- 2) выделяет необходимую область памяти и размещает в ней поля и методы объявленного экземпляра.

Шаблоны классов и наследование

Шаблоны классов могут быть наследниками как шаблонов, так и обычных классов, а также быть родителями классов и шаблонов классов.

Примеры объявлений наследников

```
// Родительский класс:  
class A  
    {...};  
  
// Шаблон класса B есть наследник класса A:  
template <class t> class B : public A  
    {...};
```

```
// Шаблон класса C есть наследник шаблона B,  
template <class W> class C: public B <W>  
{...};
```

```
// Класс D есть наследник шаблона B, указание значение параметра шаблона  
// <float> обязательно!  
class D : public B <float>  
{...};
```

Примечание. При обращении в классе наследнике к полю шаблона родительского класса следует использовать указатель на текущий класс:
this -> поле

Лабораторная работа № 1.3. Шаблоны классов

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для решения задачи Вашего варианта из ЛБ 1.2.

Требования к приложению

Следует объявить

1) шаблон класса-родитель для:

- объявления двумерного $m \times n$ массива A_T элементов типа T ;
- инициализации массива A_T случайными элементами числами T числами из отрезка $[A; B]$;
- вывода элементов массива на экран;

2) шаблон класса-наследник, содержащий метод для решения задачи Вашего варианта лабораторной работы 1.2 над массивом, который является полем родительского класса.

Содержание отчета

1. Формулировка задания, включая Ваш вариант задачи из ЛБ 1.2.
2. Расчётная часть
 - результаты выполнения задания для массивов типов `int` и `float`;
 - диаграмма классов (создание диаграмм классов см. стр. 60);
 - текст консольного приложения с необходимыми комментариями.
3. Заключение.
4. Список использованной Вами литературы.

Рекомендации по программированию

Текст консольного приложения с необходимыми комментариями

```
#include <iostream>  
#include <Windows.h>  
#include <stdlib.h> // для функций srand и rand,  
#include <ctime> // для функции time().  
using namespace std;
```

// Шаблон родительского класса:

template <class T> class Father

{

public:

T A, B; // концы отрезка.

int nRows, nCols; // число строк и столбцов.

T ** Ar; // динамический двумерный массив.

// Конструктор по умолчанию:

Father(int n = 5, int m = 7, float a = -20.0, float b = 20.0)

{

nRows = n; nCols = m; A = (T) a; B = (T) b;

// Объявление двумерного динамического массива:

Ar = new T * [nRows];

for (int k = 0; k < nRows; k++) Ar[k] = new T[nCols];

// Установка начального случайного числа:

srand(time(0));

// Инициализация двумерного динамического массива:

for (int i = 0; i < nRows; i++)

for (int k = 0; k < nCols; k++)

{

// псевдо случайное вещественное число из отрезка [0, 1):

float t = (float) (rand() % 100) / 100;

Ar[i][k] = (T) (B - A) * t + A;

}

}

~Father() // Деструктор.

{

delete[] Ar; // Освобождаем память.

}

void outAr() // Вывод элементов массива:

{

cout << "\n Массив Ar \n";

for (int i = 0; i < nRows; i++)

{

for (int k = 0; k < nCols; k++) cout << Ar[i][k] << " ";

cout << "\n";

}

}

}; // завершение объявления класса Father.

```

// Шаблон – наследник шаблона:
template <class T> class Son : public Father <T>
{
public:
    // В конструкторе класса-наследника явно обращаемся
    // к конструктору класса-родителя:

    Son <T>(int n = 5, int m = 7, T a = 1, T b = 5) : Father <T>(n, m, a, b) {}

    T Sum() // Сумма элементов массива Ar – решение задачи Варианта 0.
    {
        T s = 0;
        for (int i = 0; i < this-> nRows; i++)
            for (int k = 0; k < this-> nCols; k++)
                s += this-> Ar[i][k];
        return s;
    }
}; // завершение объявления класса Son.

int main()
{
    SetConsoleCP(1251); // установка кодировки 1251 для ввода в консоль,
    SetConsoleOutputCP(1251); // установка кодировки 1251,
    Son <int> s(2,2);
    s.outAr();
    cout << "\n" << s.Sum();
    cout << "\n\n"; system("pause");
    return 1;
}

```

1.6. Исключения и их обработка

«Любой код, сталкивающийся с исключительной ситуацией, с которой он не в силах справиться самостоятельно, генерирует исключение «в надежде» на то, что с ней совладеет некоторый другой обработчик (расположенный где-то в другом месте программы)»

Б. Страуструп

Исключение (исключительная ситуация) – это непредвиденная или аварийная ситуация, которая может возникнуть во время выполнения программы.

Исключения C++ не поддерживают обработку асинхронных событий, то есть событий, которые нарушают обычный порядок выполнения программы, например, ошибки оборудования или обработка прерываний (нажатие клавиш Ctrl + C).

Механизм исключений предназначен только для событий, которые явно вызываются программистом программе.

Обработка исключений в C++ использует операторы **try** (пробовать), **catch** (ловить) и **throw** (бросать) со следующим синтаксисом:

```
try {тело – операторы C++}  
// в тело оператора try помещают ту часть программы, в которой могут  
// произойти исключения.  
  
throw [ выражение ]  
// операторы throw в теле try генерируют исключения.  
  
catch(тип параметр) // если параметр используется в теле;  
    {тело}  
catch(тип) // если важен только тип параметра;  
    {тело}  
catch(...) // перехватываются все исключения:  
    {тело}  
// в операторах catch осуществляется обработка исключений.
```

Обработка исключений

1. Если в теле try генерируется исключение оператором throw [выражение], то управление передаётся в тот из операторов catch (тип параметр | параметр), тип параметра которого совпадает с типом выражения, а если таких нет, то оператору catch(...).
2. После выполнения соответствующего оператора catch управление передаётся оператору, который непосредственно следует за последним оператором catch.
3. Если в try исключение не генерируется, то после выполнения тела try управление передаётся оператору, который непосредственно следует за последним оператором catch.

Важно:

- ✓ операторы catch должны непосредственно следовать за оператором try;
- ✓ оператор throw должен выполняться либо внутри тела try, либо в функциях, прямо или косвенно вызываемых из тела try;
- ✓ если в try генерируется исключение, для которого не нашлось оператора catch, то происходит аварийное завершение выполнения программы.

Исключения в конструкторах и деструкторах

1. Конструкторы и деструкторы не возвращают значений, поэтому механизм исключений является наиболее приемлемым средством передачи сообщений из них, например, сообщений об ошибках.
2. Если в конструкторе экземпляра генерируется исключение, то автоматически вызывается деструкторы для полностью созданных к этому моменту в этом блоке объектов, а также для полей текущего экземпляра, являющихся полями его предков.

3. Если объект экземпляра создаётся в динамической памяти операцией new, то при возникновении исключения память из-под объекта будет корректно освобождена.

Пример генерации и обработки исключений

```
// Исключения трёх типов генерируются в функции f
float f(int a, int b)
{ if(b == 0) throw (float) 0.0;      // 1) типа float,
  if(a<0) throw -1;                 // 2) типа int,
  if(a>b) throw "a>b - ошибка";    // 3) типа строка char *.
  return (float) a*100/b;  // исключений нет.
}

void main()
{ int a, b; cout<<"\n a="; cin>>a;
  cout<<"\n b="; cin>>b;

  // В теле try вызывается функция f, в теле которой генерируются исключения:
  try {std::cout << f(a, b) << "%";}

      // Перехваты исключений:
  //с параметром типа float:
  catch(float X) {std::cout<<"\n b = "<<X;}

  // с параметром типа int:
  catch (int) {std::cout<<"\n a < 0";}

  // с параметром типа char*:
  catch(char *Str)
    {std::cout << "\n " << Str << " - Что делать не знаю";}
  // перехват всех возможно оставшихся не обработанных исключений:
  catch(...) {std::cout<<"\n  Что-то не так с a";}

  // Продолжение тела функции main:
  ...
  std::cout<<"\n\n"; system("pause");
}
```

1.7. Перегрузка операций

C++ позволяет переопределять (иначе, перегружать) все операции кроме:

. .* ?: :: # sizeof

для того, чтобы использовать знаки стандартных операций для экземпляров типа класс.

Перегрузка операций есть пример полиморфизма и осуществляется с помощью *функций-операций*:

тип operator знак_операции (список аргументов) {тело функции};

Ограничимся объявлением функции-операции метод класса.

Ограничения на перегрузку операций:

- при перегрузке операций должно сохраняться число операндов, приоритеты операций и порядок их выполнения (слева-направо или справа-налево), определённые для стандартных типов;
- если операция перегружается методом класса, то одним её аргументом должно быть поле класса;
- для стандартных типов перегружать операции нельзя;
- функции-операции наследуются (кроме операции =).

Перегрузка унарных операций методами класса

Унарная функция-операция, объявляемая методом класса, не должна иметь аргументов, так как её операндами являются поля класса.

Пример перегрузки операции прединкремента методом класса

<pre>class A {public: int N, *Ar; // аргументом является поле N: void operator ++() { for(int i = 0; i < N; i++) ++Ar[i]; } };</pre>	<pre>void main() { A a; // вызов перегруженной операции: ++a; a.out(); ... }</pre>
---	---

Пример перегрузки операции постинкремента методом класса

Функция-операции, перекрывающая постинкремент или постдекремент, должны иметь *аргумент со значением типа*, чтобы отличить их от префиксных операций:

```
void operator ++(int)
{for(int i = 0; i<N; i++) Ar[i]++;}
```

Перегрузка бинарных операций методами класса

Бинарная функция-операция, определяемая внутри класса, должна иметь один аргумент, который задаёт *второй операнд* перегружаемой операции, первый – это поле класса.

Перегрузка бинарной операции *= методом класса

```
class A
{ public: int N, *Ar;
// перегрузка операции *=,
// результат которой – массив:
int * operator *= (A a)
{ for(int i=0; i<N; i++)
  Ar[i] *= a.Ar[i];
  return Ar;
} ...};
```

```
void main()
{ A a(5), b(5);
  a *= b; // вызов перегруженной операции,
  ...
}
```

Примечание. Объявили экземпляры:
A a(5), b(5);
что бы избежать ошибки – выход индекса массива за границы диапазона.

Перегрузка операции индексирования

Операция индексирования [] обычно перегружается для классов, в которых есть поля, для которых доступ по индексу имеет смысл.

Операция индексирования должна возвращать ссылку на тип элементов, например, массива.

Пример перегрузки операции индексирования методом класса

```
class A
{ public: int N, *Ar;
...
int& operator [ ] (int k)
{ if(k < 0 || k > N-1) {cout << "\n неверный индекс";
  cout<<"\n\n"; system("pause"); exit(0);
}

  return Ar[k];}
};
void main()
{A a(5);
  cout<<"\n"<<a[0]; // нет ошибки,
  cout<<"\n"<<a[7]; // ошибка, 7 > 4.
...
}
```

Лабораторная работа № 1.4. Перегрузка операций.

Обработка исключения

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для решения задачи Вашего варианта.

Требования к приложению

Следует объявить класс:

- поля класса предназначены для входных данных,
- в конструкторе класса инициализируются поля исходными значениями,
- перегрузка унарной/бинарной операции решает задачу Вашего варианта,
- при решении задачи проверяются входные данные и генерируется исключение в случае их недопустимости; например, недопустимыми являются данные, для которых не существует решение задачи.

Содержание отчета

1. Формулировка задания.
2. Теоретическая часть (решение и алгоритм решения задачи Вашего варианта).
3. Расчётная часть
 - результаты выполнения задания;
 - текст консольного приложения с необходимыми комментариями по использованию классов, перегрузке операций, обработке исключений;
4. Заключение.
5. Список использованной Вами литературы.

Варианты заданий

1. Три точки на плоскости заданы своими координатами. Проверить существует ли треугольник с вершинами в этих точках.
2. Вычислить площадь треугольника по формуле Герона.
3. Треугольник ABC задан координатами своих вершин. Вычислить угол между высотой CD и медианой BM .
4. Вычислить площадь прямоугольного треугольника, заданного координатами своих вершин.
5. Проверить на равенство два треугольника, заданных координатами своих вершин.
6. Решить линейное уравнение $ax + b = 0$.
7. Решить квадратное уравнение $ax^2 + bx + c = 0$.
8. Вычислить расстояние между двумя точками на плоскости.
9. Вычислить расстояние от точки до прямой.
10. Вычислить расстояние между двумя плоскостями.
11. Вычислить расстояние между двумя скрещивающимися прямыми.
12. Две прямые на плоскости заданы уравнениями. Определить являются ли они параллельными.
13. Две прямые на плоскости заданы общими уравнениями. Определить являются ли они перпендикулярными.
14. Две прямые на плоскости заданы общими уравнениями. Найти угол между ними.
15. Найти точки пересечения прямой и окружности на плоскости.

1.8. Потокковые классы

Поток – это последовательность байт, передаваемая от *источника* к *приёмнику*.

Определены два действия с потоками: *чтение потока* (извлечение из потока) и *запись в поток* (*включение в поток*), которые называются *обменом с потоком*.

По направлению обмена потоки бывают:

- ✓ входные (в память);
- ✓ выходные (из памяти);
- ✓ двунаправленные (в / из памяти).

По виду устройств, с которыми работает поток, потоки бывают:

- ✓ стандартные – для передачи данных от клавиатуры на дисплей;
- ✓ файловые – для обмена данными с файлами на внешних носителях;
- ✓ строковые – для работы со строками.

Поддерживает потоки в C++ иерархия классов, построенная на двух родительских классах:

- ✓ ios – содержит объявления полей и методов общих для ввода и вывода;
- ✓ streambuf – обеспечивает взаимодействие потоков с физическими устройствами.

От этих классов наследуются классы, реализующие:

- ✓ стандартные двунаправленные потоки - iostream,
- ✓ двунаправленные файловые потоки - fstream,
- ✓ двунаправленные строковые потоки - stringstream.

Объявления классов потоков находятся в соответствующих заголовочных файлах, например, класс iostream объявлен в файле <iostream>.

Стандартные потоки

В заголовочном файле <iostream> объявлены 4 экземпляра:

- ✓ cin – ввод с клавиатуры,
- ✓ cout – вывод в консоль,
- ✓ cerr и clog – вывод на консоль сообщений об ошибках.

В классе iostream операции чтения из потока и запись в поток объявлены *перегрузками операций сдвига вправо >> и влево <<* соответственно.

Чтение и запись объявляются как операции, что позволяет использовать несколько операций в одном выражении:

```
cin >> t >> t1;    cout << t << " "<< t1;
```

Так как при перегрузке операций их приоритеты не меняются, то в случае необходимости следует использовать скобки.

Значения при записи в поток должны разделяться пробелами, знаками табуляции или перевода строки. Чтение прекращается, если очередной символ оказался недопустимым.

Пример.

```
int t, q;  
cin >> t; // набрав: 123a45, получим t = 123,  
cin >> t >> q; // набрав: 12 34 56, получим t =12, q = 34.
```

Форматирование данных

Формат данных – это порядок расположения символов данных, позволяющий распознавать их составные части (ГОСТ 17657).

В потоковых классах форматирование выполняется с помощью: *флагов*, *манипуляторов* и *форматирующих методов*. Ограничимся рассмотрением манипуляторов, которые наиболее просты при использовании.

Манипуляторы

- ✓ *Манипулятор* – это функция для форматирования данных.
- ✓ Манипуляторы объявлены в файле `<iomanip>`.
- ✓ Манипуляторы бывают простыми (без аргументов) и параметризованные (с аргументами).

Простые манипуляторы:

`endl` - при выводе включает в поток символ `\n` и выгружает буфер;
`fixed` - печать вещественных чисел в форме с фиксированной точкой;
`scientific` - печать вещественных чисел в виде мантиссы с порядком;
`showpoint` - вывод десятичной точки и дробной части.

Параметризованные манипуляторы

`setfill(char)` - установка символа-заполнителя с кодом, равного параметру;
`setprecision(int)` - установка максимального числа цифр в дробной части числа (флаг `fixed`), или общее количество значащих цифр (флаг `scientific`);
`setw(int)` установка максимальной ширины поля вывода.

Пример использования манипуляторов

```
double b = 12.45e-23;  
cout<<setw(10)<<scientific<<b;           // в консоли: 1.245000e-022  
cout <<setw(15)<<setfill('.')<<showpoint<<b; // в консоли: ...1.245000e-22
```

1.9. Файловые потоки

Файловый поток (или, просто, *файл*) – это поименованная последовательность байт на внешней памяти, например, на диске.

Определены два действия с файлами: *чтение* и *запись*.

Файлы по способу доступа к их содержимому бывают:

- ✓ последовательного доступа – чтение и запись возможна только с начала файла и последовательно с первого байта,
- ✓ прямого доступа – чтение и запись возможна с любого байта.

Ниже рассматриваются файлы последовательного доступа.

Чтение и запись в файл осуществляются методами класса `fstream`, объявленного в заголовочном файле `<fstream>`.

Этапы работы с файлами в программе

- 1) Объявить экземпляр файлового класса и связать его с файлом на диске (конструкторы).
- 2) Осуществить обмен (перегруженные операции `>>` и `<<`);
- 3) Закрыть файл (метод `close()`).

Примечание. При обмене данных файлы рассматриваются как последовательности байт. Следовательно, следует самим заботиться о правильности записи/чтения данных.

Отметим метод `eof()`, который возвращает `true`, если достигнут конец файла, и `false` в противном случае.

Примечание. Заккрытие файла осуществляется деструктором при выходе экземпляра файлового потока из области видимости. Однако, рекомендуется закрывать файл сразу после того, как необходимость в нем отпала.

Пример файлового обмена

Создать на диске папку, например, d:\QQ

```
#include <Windows.h>
```

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

// Запись в файл.

```
void writeToFile(char path[])
```

```
{ // Открываем файл для записи, если его не было, то будет создан !.
```

```
    ofstream fout(path);
```

```
    if (fout.is_open())
```

```
    { // Пишем в файл:
```

```
        for (int i = 1; i < 5; i++)
```

```
            fout << (float)i / 7 << endl;
```

```
        // Закрываем файл:
```

```
        fout.close();
```

```
    }
```

```
    else cout << "\n Ошибка открытия файла для записи.";
```

```
}
```

// Чтение файла.

```
void readFromFile(char path[])
```

```
{ // Попытка открыть файл для чтения:
```

```
    ifstream fin(path);
```

```
    if (fin.is_open())
```

```
    {
```

```
        float x;
```

```
        // Пока не конец файл читаем файл:
```

```
        while (!fin.eof())
```

```
        {
```

```
            fin >> x; cout << "\n " << x + 1;
```

```
        }
```

```
        fin.close();
```

```
    }
```

```
    else cout << "\n Файл для чтения не найден ";
```

```
    return;
```

```
}
```

```
int main()
```

```
{
```

```
    SetConsoleCP(1251); // установка кодировки 1251 для ввода в консоль,
```

```
    SetConsoleOutputCP(1251); // установка кодировки 1251,
```

```
    char P [] = "d:\\QQ\\file.dat"; // Полное имя файла.
```

```

cout << "\n" << P;
writeToFile(P); // Пишем в файл.
readFromFile(P); // Читаем файл.
cout << "\n\n"; system("pause");
return 0;
}

```

Лабораторная работа № 1.5. Файловые операции

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для выполнения задания Вашего варианта ЛБ 1.2 с использованием файловых операций.

Требования к приложению

- в классе-родителе объявить методы для записи/чтения элементов двумерного массива в файл.

Содержание отчета

1. Формулировка задания, включая Ваш вариант задания ЛБ 1.2.
2. Расчётная часть
 - результаты выполнения задания;
 - текст консольного приложения с необходимыми комментариями по файловым операциям.
3. Заключение.
4. Список использованной Вами литературы.

Рекомендации по программированию

Использовать функции, реализующие файловый ввод/вывод, объявленные на стр. 27.

1.10. Строковые потоки

В языке C++ со строками можно работать как с массивами символов. Это быстро, но чревато выходом индекса за границы строки с последующим зависанием программы.

При работе со строковыми потоками выйти за границы строки невозможно, но проигрываем в скорости.

Строковые потоки поддерживаются классом `string`, который объявлен в заголовочном файле `<string>`.

Объявление экземпляров классов `string` (конструкторы)

```

#include <string>
string str("символы");

```

Операции над строками

- Операция присваивания
`str = "символы ASCII с латиницей";`
- Конкатенация строк: `+` `+=`
- Лексикографическое сравнение строк: `==` `!=` `<` `<=` `>` `>=`
- Операция индексирования: `[]`, `[k]` – k-й символ строки, нумерация начинается с 0

Если индекс символа строки больше длины строки, то генерируется исключение `out_of_range`.

Размеры строк устанавливаются автоматически.

Методы класса `string` (C++)

- Длина строки `S` в символах: `S.length()`
- Вырезка подстроки:
`S1.assign(string S, int P, int N);` // `S1` = подстрока строки `S` из `N` символов,
// начиная с `P`-го.
- Вставка строки:
`S.insert(int P, string S1);` // строка `S1` вставляется в строку `S`,
// начиная с `P`-го символа.
- Замена подстроки
`S.replace(int P, int N, string S1);` // часть строки `S` из `N` символов,
// начиная с `P`-го, заменяется на строку `S1`.
- Поиск подстроки
`S.find(string S1);` // возвращает номер первого символа, с которого
// строка `S1` входит в строку `S`,
// если вхождения нет, то возвращается постоянная `npos`.
- Очистка строки
`S.clear();` // `S` = ""
`bool S.empty()` // = `true`, если `S=""`; = `false`, если `S != ""`.

Примечание. Для ввода/вывода в консоль кириллицы:

- подключить заголовочный файл:
`#include <Windows.h>`
- в теле функции `main()` установить кодировки 1251:
`SetConsoleCP(1251);`
`SetConsoleOutputCP(1251);`

Лабораторная работа № 1.6. Использование строкового типа

Задание. Создать в MS Visual Studio на языке C++ консольное приложение для решения задачи Вашего варианта.

Требование к приложению

- объявите класс, метод которого решает задачу Вашего варианта.

Содержание отчета

1. Формулировка задания, Ваш вариант задачи, требования к приложению.
2. Расчетная часть
 - результаты выполнения задания;
 - текст консольного приложения с необходимыми комментариями по использованию операций и методов класса `string`.
3. Заключение.
4. Список использованной Вами литературы.

Варианты заданий

1. В строке, найти конец предложения, обозначенный символом «точка». В следующем слове первую строчную букву заменить на прописную.
2. Для встречающихся в заданной строке пар рядом расположенных символов указать, сколько раз встречаются каждое из таких двух символьных сочетаний.
3. Отредактировать предложение, удаляя из него лишние пробелы, оставляя только по одному пробелу между словами.
4. Для каждого символа в введённом с клавиатуры слова указать, сколько раз он встречается в строке. Сообщение об одном символе должно печататься не более одного раза.
5. В заданной строке найти самое короткое и самое длинное слово.
6. В строке заменить символы пробела на символы “,”. Если в тексте встречается несколько пробелов подряд, то вместо них поставить одну запятую.
7. С клавиатуры вводится текст из прописных и строчных русских букв, и цифр. Ввод прекращается при нажатии клавиши Enter. Вывести на экран все строчные буквы введенного текста.
8. С клавиатуры вводится текст из русских букв и цифр. Ввод прекращается при нажатии клавиши Enter. Найти процентное содержание строчных букв в исходном тексте.
9. С клавиатуры вводится текст из русских букв и цифр. Ввод прекращается при нажатии клавиши Enter. Напечатать только цифры, входящие в этот текст.
10. С клавиатуры вводится текст из строчных русских букв. Ввод прекращается при нажатии клавиши Enter. Определить упорядочены ли буквы по алфавиту. Если нет, то упорядочить.
11. С клавиатуры вводится текст из строчных русских букв. Ввод прекращается при нажатии клавиши Enter. Напечатать эту строку, предварительно заменив на «ха» каждое вхождение буквы х.
12. Вычислить процентное содержание гласных букв в строке.
13. Дана последовательность, содержащая до 5 слов, в каждом из которых до 5 строчных латинских букв; между соседними словами — запятая, за последним словом — точка. Напечатать эту последовательность слов в обратном порядке.
14. В каждом слове текста заменить букву «а» на букву «е», если «а» стоит на четном месте, заменить букву «б» на сочетание «ак», если «б» стоит на нечетной позиции.
15. Проверить, можно ли из букв первого слова составить второе слово.

1.10. Контейнерные классы

Контейнерным классом или *контейнером* называется *шаблон класса* для работы с *последовательностями данных*, организованными специальным образом.

В библиотеке (STL – Standard Template Library) объявлены контейнеры для работы с основными последовательностями данных, используемых при создании программ.

Использование контейнеров:

- ✓ повышает надежность программ, их переносимость и универсальность,
- ✓ сокращает сроки их разработки,

за счет *снижения быстродействия* программ.

В C++ контейнеры представлены двумя видами: *последовательные* и *ассоциативные*:

- *Последовательные контейнеры* обеспечивают хранение конечного количества данных в виде непрерывной последовательности.

Примеры последовательных контейнеров: *векторы, стеки, очереди, списки*.

- *Ассоциативные контейнеры* обеспечивают быстрый доступ по ключу; они построены на основе сбалансированных деревьев;

Примеры ассоциативных контейнеров: *словари* и *множества*.

Каждый вид контейнера объявлен в соответствующем заголовочном файле и обеспечивает «свой» набор действий над данными.

Выбор вида контейнера зависит от того, что требуется делать с данными в программе. Например, если необходимо часто вставлять/удалять данные «из середины», то рациональнее использовать список, а, если вставка/удаление происходит в начале или в конце, то стек или очередь.

Примечание. Для краткости принято называть экземпляры контейнеров именами контейнеров, например, *экземпляры контейнера list* (список) называют *списками*.

1.10.1 Последовательный контейнер vector (вектор)

Шаблон класса `vector` - часто применяемый шаблон стандартной библиотеки STL.

Шаблон класса `vector`:

- призван заменить *управляемый программистом* динамический массив,
- обладает свойствами *односвязных списков*⁴,
- не ускоряет работу программы,
- предоставляет методы для типовых операций с массивами.

Для работы с векторами следует подключить заголовочные файлы `vector` и `iterator`, стандартное пространство имен `std`.

Примечание. В файле `iterator` объявлены так называемые *итераторы* (аналоги указателей) для обеспечения доступа к элементам векторов.

⁴ Односвязным списком называется последовательность элементов типа структура, одно из полей является указателем на элемент списка, остальные поля содержат данные.

Методы контейнера vector

```
#include <iostream>
#include <Windows.h>
# include <vector>
# include < iterator>
using namespace std;
vector <int> initV(int N)
{
    vector <int> v(N); // объявление экземпляра вектора из N элементов
    типа int.
    // Инициализация элементов экземпляра вектора.
    for (int i = 0; i < v.size(); i++)
        v[i] = i + 2;
    return v;
}
void outV(vector <int> v)
{ // Вывод элементов вектора в консоль:
    cout << "\n";
    for (int i = 0; i < v.size(); i++)
        cout << " " << v[i];
}
vector <int> insN(int a, int N, vector <int> v)
{
    cout << "\n Вставка элемента в вектор.";
    if (N >= v.size() || N <= 0)
    {
        cout << "\nВставить нельзя";
        return v;
    }
    vector <int>::iterator itV; // объявление итератора.
    if (N == 1)
    {
        cout << "\n Вставка элемента "<<a<< " первым в вектор:";
        itV = v.begin(); v.insert(itV, a);
        return v;
    }
    cout << "\n Вставка элемента "<<a<<" "<< N << "-м в вектор:";
    itV = v.begin();
    for(int i = 1; i<N; i++)
        ++itV;
    v.insert(itV, a);
    return v;
}
```

```

vector<int> del(int N, vector<int> v)
{
    cout << "\n Удаление элемента из вектора.";
    if (N >= v.size() || N <= 0)
    {
        cout << "\nУдалить нельзя";
        return v;
    }
    vector<int>::iterator itV; // объявление итератора.
    if (N == 1)
    {
        cout << "\n Удаление первого элемента:";
        itV = v.begin(); v.erase(itV);
        return v;
    }
    cout << "\n Удаление " << N << "-го элемента:";
    itV = v.begin();
    for (int i = 1; i < N; i++)
        ++itV;
    v.erase(itV);
    return v;
}

vector<int> rndInit(int N, int a, int b)
{ // Инициализация вектора случайными числами из отрезка [a, b].
  srand(time(0)); // // установка начального случайного числа,
  vector<int> v(N); // объявление экземпляра вектора из N элементов типа int,
    for (int k = 0; k < N; k++)
    {
        // случайное целое число из отрезка [a, b],
        int num = rand() % (b - a + 1) + a;
        v[k] = num;
    }
    return v;
}

```

1.10.2 Ассоциативный контейнер map (словарь)

Словарем (ассоциативным массивом или отображением) называется совокупность пар (*ключ, элемент*), причем:

- ключи предназначены для идентификации элементов,
- ключи и элементы могут быть любого типа.

Примеры словарей:

- толковый словарь русского языка – значениями *ключей* являются русские слова, а *значениями* элементов – их толкования, например, («*абы*», «*лишь бы*, *только бы*, *делает кое-как*, *скорее закончить*»)

- массивы – значениями ключа являются неотрицательные целые числа (индексы).

В заголовочном файле <map> библиотеки STL объявлены два вида словаря: map (*словарь* - значения ключа уникальны) и multimap (*словарь с дубликатами* - значения ключа могут повторяться).

Методы контейнера map

```
#include <iostream>
#include <Windows.h>
#include <iterator>
#include <map>
#include <string>
using namespace std;

typedef std::map<std::string, std::string> myMap;
// typedef map <string, string> myMap; // обозначение типа словаря,
// словарь состоит из пар экземпляров класса string,

void outM(myMap m)
{ // Вывод словаря в консоль:
    myMap::iterator it;
    if (m.begin() == m.end())
        cout << "\n Словарь пустой";
    else
        for (it = m.begin(); it != m.end(); it++)
            cout << "\n " << it->first << " " << it->second;
}

myMap insP(string key, string val, myMap m)
{ // Вставка пары (key, el) в словарь.
    // используется перегруженная операция индексирования [].
    m[key] = val;
    return m;
}

myMap createM(myMap m)
{ // создание словаря:
    string key, el;
    key = "аще"; el = "Бог"; m[key] = el;
    key = "по"; el = "нас"; m[key] = el;
    key = "никто"; el = "же"; m[key] = el;
    key = "на"; el = "ны"; m[key] = el;
    return m;
}
```

```

void searchByKey(string key, myMap m)
{ // Поиск в словаре элемента по ключу
    myMap::iterator it;
    for (it = m.begin(); it != m.end(); it++)
        if (it->first == key)
        {
            cout << "\nВ словаре есть элемент " << it->second << " с
ключом " << it->first;
            return;
        }
    cout << "\n В словаре нет элемента с ключом " << key;
    return;
}

myMap eraseP(string key, myMap m)
{ // Удаление из словаря пары(ключ, элемент) по ключу
    myMap::iterator it;
    if (m.begin() == m.end())
    {
        cout << "\n Словарь пустой";
        return m;
    }
    for (it = m.begin(); it != m.end(); it++)
        if (it->first == key)
        {
            m.erase(it);
            cout << "\nПара с ключом " << key << " удалена";
            return m;
        }
    cout << "\n Пары с ключом " << key << " в словаре нет";
    return m;
}

```

Глава 2. Объектно-ориентированные системы программирования

2.1. Создание приложений windows forms в MS VS 2019/2022

Приложением с графическим интерфейсом пользователя⁵ (GUI-приложением), называется программа, работающая под управлением операционной системы (ОС) Windows.

Это управление заключается в том, что ОС отслеживает наступление событий при выполнении программы и вызывает (передает управление) функцию, операторы которой должны по замыслу программиста выполняться как реакция на произошедшее событие.

⁵ Графический интерфейс пользователя (GUI) – это способ взаимодействия человека с компьютером через визуальные элементы, а не текстовые команды.

Примеры событий, наступление которых фиксирует ОС: щелчок мышью по окну, перетаскивание окна по экрану, истечение определённого промежутка времени и т.д.

Одним из видов GUI-приложений, которые можно создавать в MS VS является приложение *windows forms*.

Приложения *windows forms* при выполнении лабораторных работ будем создавать на языке C# – это современный язык с C++ подобным синтаксисом, разработанный специально для платформы .NET с акцентом на безопасность и производительность разработчика.

Создавать приложения *windows forms* на языке C# в MS VS 2019/2022 существенно проще чем на C++.

Создаваемое в MS VS приложение *windows forms* «собирается» из нескольких файлов, поэтому называется проектом. Результатом компилирования этого проекта и является приложение *windows forms*.

Решением в MS VS называется контейнер, содержащий один или несколько связанных приложений *windows forms*.

Проект создаётся из *элементов*, основным из которых является *форма*.

Используемая терминология:

- *элемент* означает и класс, например, Button, и экземпляр класса, например, button1; при изображении на форме элементы автоматически нумеруются;
- *свойство элемента* – поле экземпляра;
- *обработчик события элемента* – метод экземпляра, который объявляется программистом, а вызывается операционной системой;
- *метод элемента* – метод экземпляра, который объявляется программистом и вызывается из программы.

Следует подчеркнуть, что элементы, как экземпляры, объявляются типа класс, поэтому обращение к их полям, методам и обработчикам событий осуществляется посредством операции точка (.).

Создание приложений windows forms

Интерфейс MS VS 2019/2022 представлен на рис. 1.

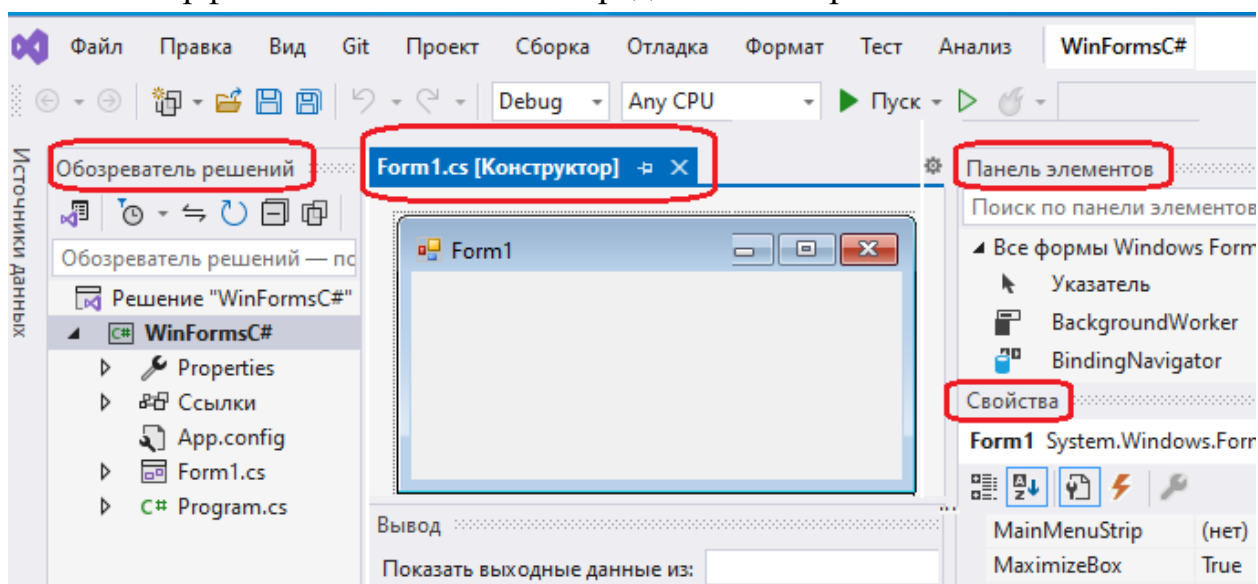


Рисунок 1

Приложения windows forms в MS VS на C# создаются из перетаскиваемых элементов из Панели элементов в окно Form1.cs[Конструктор]», свойства которых изменяются в окне Свойства на закладках , , , а их обработчики событий объявляются в окне Свойства на закладке .

Изменение свойств элементов на форме:

- [Л] элемент;
- в окне «Свойства»: [Л] , либо [Л] , либо [Л] .
- [Л] свойство – выбрать/набрать значение.

Объявление обработчиков событий для элементов:

- [Л] элемент;
- в окне «Свойства»: [Л] .
- [Л] имя события – [Л]² имя события;
- в окне Form1.Designer.cs набрать тело обработчика в «заготовке»:

```
private void элемент_Событие(object sender, EventArgs e)
{
    groupBox2.Visible = true; // пример тела обработчика события.
}
```

2.2. Основные элементы для создания приложений windows forms

Общие свойства элементов

BackColor – цвет фона элемента, ForeColor – цвет шрифта надписи, Enabled – доступность элемента (true – доступен, false – не доступен).

2.2.1 Форма - - контейнер для элементов

свойство: Text – имя окна приложения windows forms.

2.2.2 MenuStrip – меню

- создание полосы пунктов меню.

1. Вставка элементов MenuStrip (*их допускается несколько*) в форму:

- открыть категорию «Меню и панели инструментов»
[Л] MenuStrip - [Л] по форме, (рис. 2)
- [Л] в окнах «Вводить здесь» открывает редактор для набора названия пунктов/подпунктов меню.

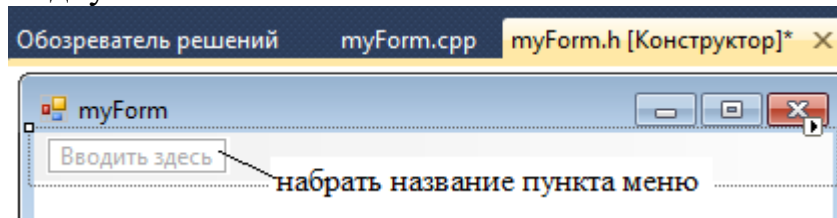


Рисунок 2

Каждый пункт/подпункт меню становится элементом со своими свойствами и обработчиками событий.

2. Объявление обработчиков событий для элементов:

- [Л] по элементу на форме;
- в окне «Свойства»: выбрать обработчик события
- набрать тело обработчика в «заготовке»:

```
private void элемент_Событие(object sender, EventArgs e)
{
    тело обработчика.
}
```

Пример «заготовки» обработчика события «выбран пункт меню»

```
private void лБ22ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox1.Visible = true;
}
```

Лабораторная работа № 2.1. Windows forms. Элементы форма и меню
Задание. Создайте в MS VS приложение windows forms на языке C++, состоящим из формы и меню.

Требования к проекту

- измените заголовок окна формы на:
"ООЯиСП – Курсовой проект. Автор ФИО. Гр. ХХГ1"
- добавьте элемент меню (menuItem) с одним пунктом ЛБ 2.2

Содержание отчета

1. Формулировка задания, требования к проекту.
2. Скриншот открытого приложения windows forms.

Рекомендации по выполнению задания

Изменение свойств элементов (стр. 37), вставка полосы меню (стр. 38).

Скриншот отрытого приложения windows forms приведён на рис. 3.

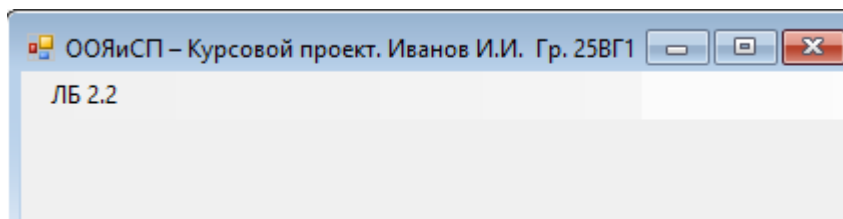


Рисунок 3

2.2.3 Элементы Метка, Кнопка, Редактор, Рамка с надписью

Label (метка) – вывод короткого текста.

свойство: Text – надпись на метке,

событие: Click – щелчок по метке.

Button (кнопка) – дать старт чему-либо, начать чего-либо.

свойство: Text – надпись на кнопке,

событие: Click – щелчок по кнопке.

TextBox (редактор) – ввод и редактирование нескольких строк текста.

свойство: Text – текст в окне редактора.

GroupBox (рамка с надписью) - контейнер для элементов.

свойство: Text – надпись на рамке.

Лабораторная работа № 2.2. Windows forms.

Элементы «кнопка», «редактор»

Задание. Добавьте к созданному при выполнении ЛБ 2.1 приложению windows forms элементы для решения задачи Вашего варианта.

Требование к проекту

- 1) Для ввода строки использовать элемент textBox1.
- 2) Для вывода результата решения задачи использовать элемент textBox2.
- 3) Решение задачи объявить в обработчике события «нажатие кнопки».
- 4) Все используемые элементы разместить в контейнере groupBox1.
- 5) Начальное значение свойства groupBox1->Visible = False, выбор пункта меню «ЛБ 2.2» устанавливает это значение в True.

Содержание отчета

1. Формулировка задания и содержание отчёта.
2. Расчетная часть
 - состав проекта;
 - обоснование выбора элементов и измененные их свойства;
 - объявления обработчиков событий с необходимыми комментариями;
 - результат тестирования.
3. Заключение.
4. Список использованной Вами литературы.

Варианты заданий

1. В строке найти самое короткое и самое длинное слово.
2. В строке заменить символы пробела на символы “,”. Если в тексте встречается несколько пробелов подряд, то вместо них поставить одну запятую.
3. С клавиатуры вводится строка из прописных и строчных русских букв, и цифр. Ввод прекращается при нажатии клавиши Enter. Вывести на экран все строчные буквы введенной строки.
4. С клавиатуры вводится строка из русских букв и цифр. Ввод прекращается при нажатии клавиши Enter. Найти процентное содержание строчных букв в исходной строке.
5. С клавиатуры вводится строка из русских букв и цифр. Ввод прекращается при нажатии клавиши Enter. Вывести на экран только цифры, входящие в эту строку.
6. С клавиатуры вводится строка из строчных русских букв. Ввод прекращается при нажатии клавиши Enter. Определить упорядочены ли буквы по алфавиту. Если нет, то упорядочить.
7. С клавиатуры вводится строка из строчных русских букв. Ввод прекращается при нажатии клавиши Enter. Напечатать эту строку, предварительно заменив на «ха» каждое вхождение буквы х.
8. Вычислить процентное содержание гласных букв в строке.
9. Дана строка из слов, между словами – запятая, за последним словом - точка. Напечатать эти слова в обратном порядке.
10. В каждом слове строки заменить букву «а» на букву «и», если «а» стоит на четном месте, заменить букву «б» на сочетание «ак», если «б» стоит на нечетной позиции.
11. Проверить, можно ли из букв первой строки составить вторую строку.
12. Отредактировать строку, удаляя из неё лишние пробелы, оставляя только по одному пробелу между словами.
13. Для встречающихся в заданной строке пар рядом расположенных символов указать, сколько раз встречаются каждое из таких двух символьных сочетаний.
14. В строке, найти конец предложения, обозначенный символом «точка». В следующем слове первую строчную букву заменить на прописную.
15. Для каждого символа, введенного с клавиатуры строки указать, сколько раз он встречается в строке. Сообщение об одном символе должно печататься не более одного раза.

Рекомендации по программированию

Вариант 0. В строке заменить цифры на символы «*».

Состав проекта приведён на рис. 4.

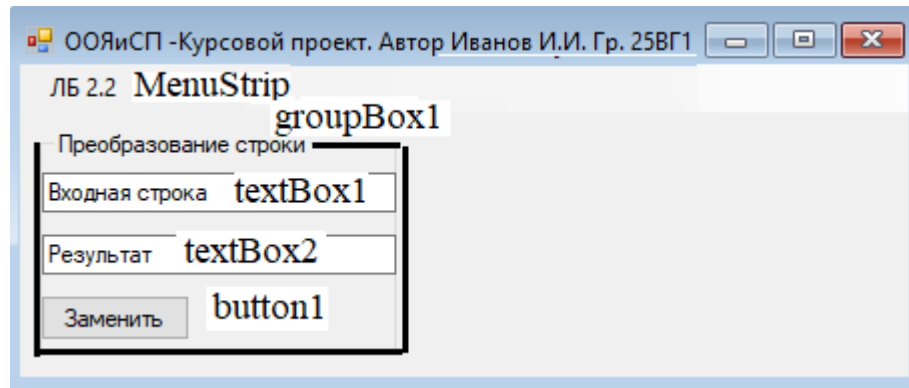


Рисунок 4

Обоснование выбора элементов и измененные их свойства

MenuStrip – меню с одним пунктом.

groupBox1 – контейнер для элементов⁶,

groupBox1->Text = "Преобразование строки"

groupBox1->Visible = False

textBox1 – для ввода строки,

textBox1->Text = "Входная строка"

textBox2 – для вывода результат преобразования строки,

textBox2->Text = "Результат"

button1 – для решения задачи Варианта 0,

button1->Text = "Заменить"

Объявления обработчиков событий

Объявление обработчиков событий элементов см. стр. 37.

```
private void лБ22ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox1.Visible = true;
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    // [Л] button1 – решение задачи.
    // Используем методы класса string (C#)
    string Q = textBox1.Text;
    if (Q.Length > 0) // если строка Q не пустая,
    {
        string T = "", q;
        // посимвольный просмотр Q:
        for (int i = 0; i < Q.Length; i++)
```

⁶ Вставка элементов в контейнер: изменить размеры контейнера, [Л] по элементу на Панели элементов, [Л] в контейнере.

```

{
    q = Q.Substring(i, 1); // q – i-й символ строки,
    int k = q.CompareTo("0"),
    m = q.CompareTo("9");
    if (k >= 0 && m <= 0) // если q – цифра,
        T += "*"; // замена цифру на *,
    else T += q; // иначе q – не цифра, добавляем q результату.
}
textBox2.Text = T;
}
}

```

Результат тестирования представлен на рис. 5.

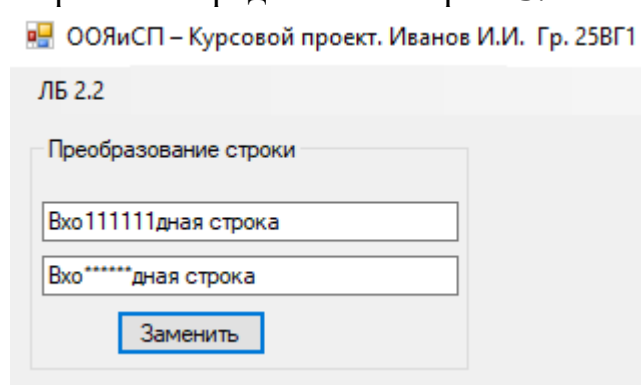


Рисунок 5

2.2.4. Элемент Список

ListBox (список) - одномерный массив элементов типа string.

Основные методы:

Items[i] – i-ая строка,

Items->Add(строка) – внесение строки последней в список,

Items->Clear() – очистка списка,

Items->Count – количество элементов в списке (нумерация с 0),

Items->Insert(индекс, строка) – вставка в список строки по указанному индексу,

Items->Remove(строка) – удаление из списка указанной строки,

Items->RemoveAt(индекс) – удаление из списка строки по указанному значению её индекса,

SelectedIndex – возвращает/задаёт индекс выбранной строки,

SelectedItem – возвращает/задаёт выбранную строку.

Лабораторная работа № 2.3. Windows forms.

Элемент Список. Контейнер вектор

Задание. Добавьте к приложению windows forms, созданному при выполнении ЛБ 2.2 элементы для решения задачи Вашего варианта.

Требование к проекту

1. Элементы входного целочисленного массива следует представить строками элемента textBox.
2. Число элементов массива вводится интерактивно; элементы массива инициализируются целыми случайными числами из отрезка $[-100-N; 100+N]$, где N – номер Вашего варианта.
3. Для решения задачи добавить в проект элементы:
 - listBox1 – для отображения элементов входного массива;
 - listBox2 – для отображения элементов массива - результата решения задачи Вашего варианта;
 - label1->Text = "N=";
 - textBox3->Text = "10" – для интерактивного ввода числа элементов вектора;
 - button2 – для вывода входного вектора, решения задачи и вывода результата;
 - все элементы следует поместить в контейнер groupBox2 (groupBox2 -> Visible=false);
 - пункт меню "ЛБ 2.3", выбор которого скрывает делает видимым groupBox2.

Содержание отчета

1. Формулировка задания.
2. Расчетная часть
 - добавленные в проект элементы;
 - объявление класса;
 - обоснование выбора элементов и измененные их свойства;
 - объявления обработчиков событий с необходимыми комментариями;
 - результаты тестирования.
3. Заключение.
4. Список использованной Вами литературы.

Варианты заданий

1. Отсортировать элементы массива с нечетными индексами по не убыванию, а с четными – по не возрастанию.
2. Сколько в массиве положительных элементов с простыми индексами.
3. Найти наибольший элемент среди элементов с простыми индексами.
4. Найти наименьший элемент среди элементов с простыми индексами.
5. Удалить из массива каждый второй элемент.
6. Образовать новый массив из сумм соседних элементов.
7. Заменить в массиве отрицательные элементы нулями.
8. Сколько в массиве пар соседних равных элементов?

9. Вывести на экран все элементы массива, которые больше своего «соседа» слева и меньше своего «соседа» справа, крайние элементы в их число не входят.
10. Сжать массив, удалив элементы, принадлежащие отрезку $[a; b]$.
11. Найти сумму цифр всех элементов целочисленного массива.
12. Вывести неповторяющиеся элементы массива.
13. Найти наибольший по модулю элемент среди уникальных по модулю элементов.
14. Найти наименьший по модулю элемент среди уникальных по модулю элементов.
15. Вывести повторяющиеся элементы массива.

Рекомендации по программированию

Вариант 0. Заменить каждый элемент массива на противоположный.

Состав проекта представлен на рис. 6.

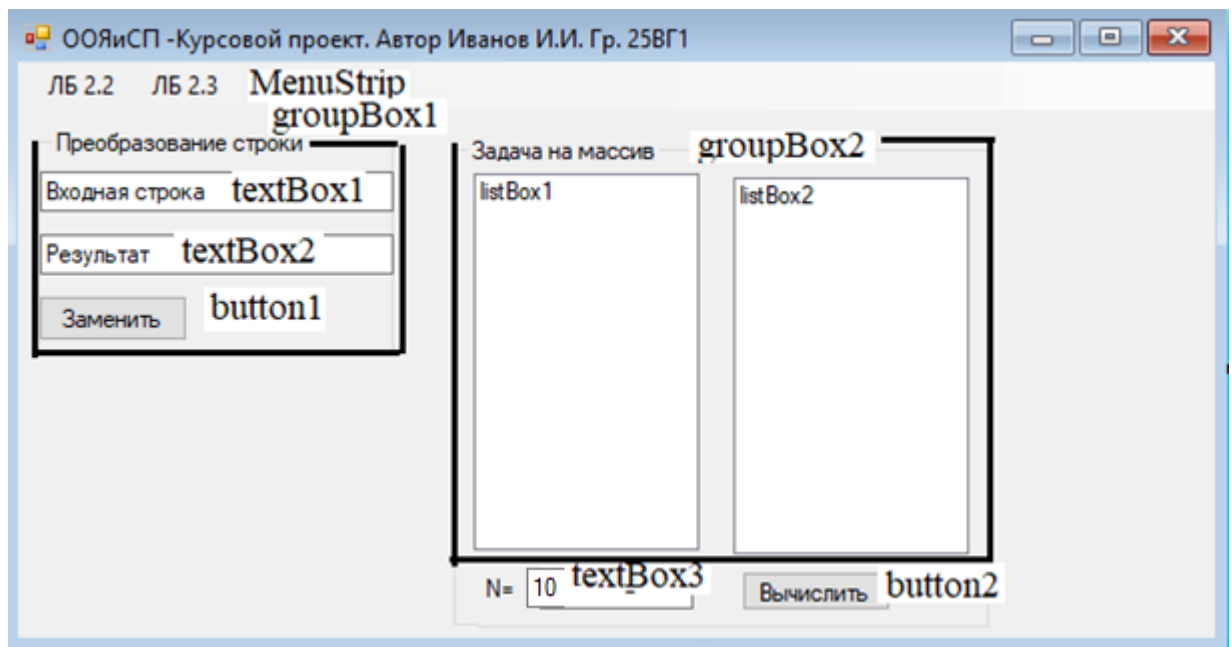


Рисунок 6

Объявление обработчиков событий

Рекомендация для решения задачи следует использовать методы контейнера vector (стр. 32).

```
private void лБ23ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox2.Visible = true;
}
```

```
private void button2_Click(object sender, EventArgs e)
{
    // Решение задачи
    string S = textBox3.Text;
    int N = int.Parse(S); // число элементов в массиве,
    listBox1.Items.Clear();
```

```

listBox2.Items.Clear();
Random random = new Random();
int a = -100, b = 20, num;

for (int i = 0; i < N; i++)
{
    num = random.Next(a, b + 1); // случайное целое число из [a, b],
    listBox1.Items.Add(num.ToString());
    listBox2.Items.Add((-num).ToString());
}
}

```

Результат тестирования приведён на рис. 7.

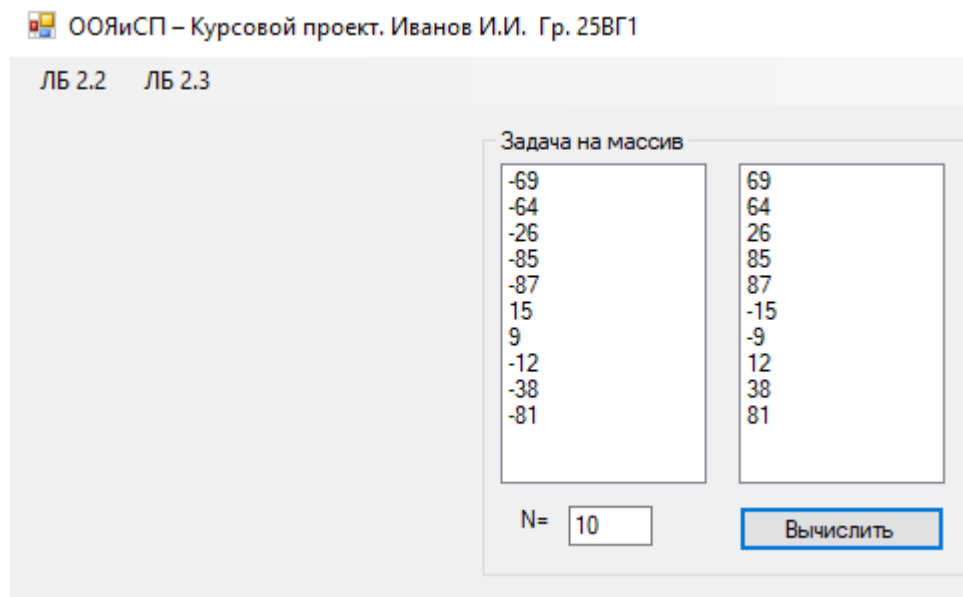


Рисунок 7

2.2.5. Элемент Таблица Данных

Элемент Таблица Данных (DataGridView) – это двумерный массив элементов типа string (нумерация строк и столбцов начинается с 0).

Основные методы элемента DataGridView (C#)

```

dataGridView1.ColumnCount = 3; // число столбцов = 3,
dataGridView1.RowCount = 3; // число строк = 3,
// сокрытие заголовков:
dataGridView1.ColumnHeadersVisible = false; // столбцов,
dataGridView1.RowHeadersVisible = false; // строк,
// установка ширины столбцов = 30:

```

```

for(int c = 0; c < dataGridView1->ColumnCount; c++)
    dataGridView1.Columns[c].Width = 30;

```

Запись/чтение чисел в/из ячейки таблицы

```

int num = 5;
dataGridView1.Rows[3].Cells[5].Value = num.ToString();
S += int.Parse(dataGridView1.Rows[i].Cells[j].Value.ToString());

```

Лабораторная работа № 2.4. Windows forms. Элемент Таблица Данных
Задание. Добавьте к приложению windows forms, созданному при выполнении ЛБ 2.3 элементы для решения задачи Вашего варианта.

Требование к проекту

1. Число строк и столбцов массива вводятся интерактивно, элементы массива инициализируются небольшими случайными целыми числами.
2. Для решения задачи добавить в проект элементы:
 - dataGridView1 – для отображения элементов двумерного массива;
 - label2->Text = "R=";
 - textBox4->Text = "5" – интерактивный ввод числа строк в таблице;
 - label3->Text = "C=";
 - textBox5->Text = "3" – интерактивный ввод числа столбцов в таблице;
 - button3->Text = "Инициал" – для заполнения таблицы элементами массива, записи элементов массива в файл;
 - button4->Text = "Решить" - для решения задачи, числа следует брать из файла ;
button4->Enabled = false // без инициализации массива решить задачу нельзя;
 - label4 – для отображения ответа, label4 -> AutoSize = False;
 - label5 -> "e:\QQ\file.dat" // имя файла для записи и чтения.
 - все указанные выше элементы следует поместить в контейнер groupBox3 (groupBox3 -> Visible = false);
 - пункт меню "ЛБ 2.4", выбор которого скрывает groupBox1 и groupBox2 и делает видимым groupBox3.

Содержание отчета

1. Формулировка задания и содержание отчёта.
2. Расчетная часть
 - добавленные в проект элементы;
 - обоснование выбора элементов и измененные их свойства;
 - объявления обработчиков событий с необходимыми комментариями;
 - результаты тестирования.
3. Заключение.
4. Список использованной Вами литературы.

Варианты заданий

Во всех вариантах массив двумерный и целочисленный.

1. Среди всех элементов массива, которые больше своего «соседа» слева и меньше своего «соседа» справа в «своей» строке, вывести на экран элемент, сумма индексов которого наименьшая.
2. Среди всех элементов массива, которые больше своего «соседа» сверху и меньше своего «соседа» снизу в «своем» столбце, вывести на экран элемент, сумма индексов которого наибольшая.
3. Найти сумму элементов массива, первые индексы которых есть простые числа, а вторые – составные.

4. Найти произведение элементов массива, оба индекса которых есть составные числа.
5. Сколько в массиве элементов, которые представимы в виде суммы двух простых чисел.
6. Сколько в массиве элементов, которые представимы в виде произведения двух простых чисел.
7. Найти сумму элементов массива, оба индекса которых есть составные числа.
8. Среди всех элементов массива, которые больше своего «соседа» слева и меньше своего «соседа» справа в «своей» строке, вывести на экран наименьший.
9. Сколько в двумерном массиве четных положительных элементов.
10. Найти наибольший среди элементов массива с чётной суммой индексов.
11. Найти наименьший элемент массива среди элементов, сумма индексов которых есть простое число.
12. Сколько в массиве нечетных отрицательных элементов.
13. Сколько в массиве четных положительных элементов.
14. Среди всех элементов массива, которые больше своего «соседа» сверху и меньше своего «соседа» снизу в «своем» столбце, вывести на экран наименьший.
15. Сколько в массиве элементов, которые представимы в виде произведения трех простых чисел.

Рекомендации по программированию

Вариант 0. Вычислить сумму элементов двумерного массива.

Состав проекта представлен на рис. 8.

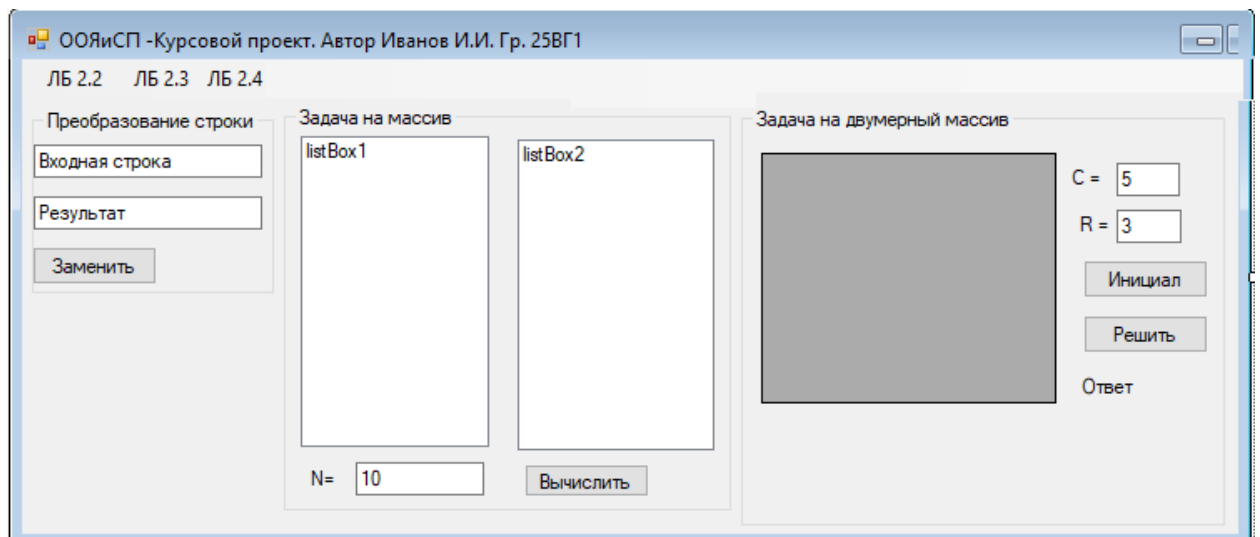


Рисунок 8

Обработчик события

```
private void лБ24ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox3.Visible = true;
}

private void button3_Click(object sender, EventArgs e)
{
    // [Л] button3
    // Инициализация таблицы
    dataGridView1.RowCount = int.Parse(textBox4.Text); // число строк,
    dataGridView1.ColumnCount = int.Parse(textBox5.Text); // число столбцов,
                                                // скрываем:
    dataGridView1.ColumnHeadersVisible = false; // заголовки столбцов,
    dataGridView1.RowHeadersVisible = false; // заголовки строк,
    // устанавливаем ширину столбцов = 30:
    for (int i = 0; i < dataGridView1.ColumnCount; i++)
        dataGridView1.Columns[i].Width = 30;
    // Отображаем элементы массива в ячейках таблицы:
    Random random = new Random();
    int a = -20, b = 20, num;
    for (int i = 0; i < dataGridView1.RowCount; i++)
        for (int j = 0; j < dataGridView1.ColumnCount; j++)
        {
            num = random.Next(a, b + 1); // случайное целое число из [a, b],
            dataGridView1.Rows[i].Cells[j].Value = num.ToString();
        }
    button4.Enabled = true; // кнопка «решить» стала доступна.
}

private void button4_Click(object sender, EventArgs e)
{
    // Решение задачи варианта 0.
    int S = 0;
    for (int i = 0; i < dataGridView1.RowCount; i++)
        for (int j = 0; j < dataGridView1.ColumnCount; j++)
            S += int.Parse(dataGridView1.Rows[i].Cells[j].Value.ToString());
    label4.Visible = true;
    label4.Text = S.ToString();
}
```

Лабораторная работа 2.5. Windows forms. Элементы OpenFileDialog и SaveFileDialog

Задание. Добавьте к приложению windows forms, созданному при выполнении ЛБ 2.4 элементы для чтения и записи элементов таблицы в файл.

Требование к проекту

1. Добавить на форму элементы openFileDialog1 и saveFileDialog1.
2. Добавить в элемент groupBox3 элементы:
 - button5->Text = "Запись" – элементов таблицы в файл;
button5->Enabled = false
 - button6->Text = "Чтение" – для чтения элементов из файла в таблицу;
button6->Enabled = false
 - в обработчике события button3_Click(object sender, EventArgs e) после
button4.Enabled = true; // кнопка «решить» стала доступна.
добавить операторы:
button5.Enabled = true;
button6.Enabled = true;

Содержание отчета

1. Формулировка задания и содержание отчёта.
2. Расчетная часть
 - добавленные в проект элементы;
 - обоснование выбора элементов и измененные их свойства;
 - объявления обработчиков событий с необходимыми комментариями;
 - результаты тестирования.
3. Заключение.
4. Список использованной Вами литературы.

Рекомендации по программированию

Состав проекта представлен на рис. 9.

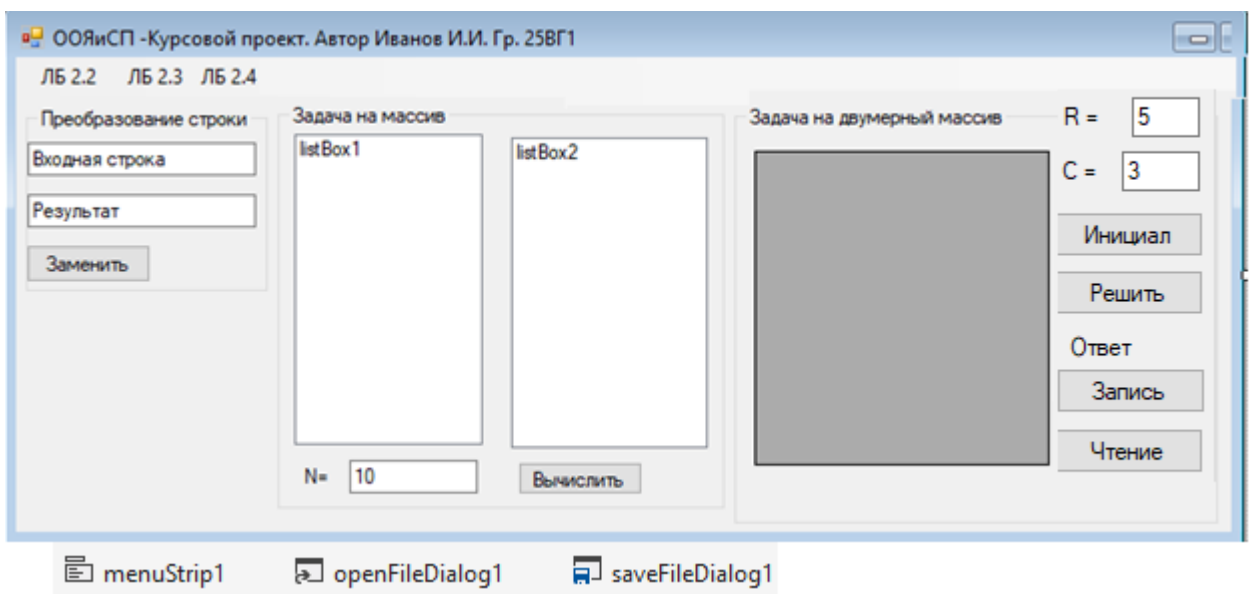


Рисунок 9

Обработчик события

```
private void button5_Click(object sender, EventArgs e)
{ // [Л] button5 “Запись”.
  if (openFileDialog1.ShowDialog() == DialogResult.OK)
  {
    string name = openFileDialog1.FileName; // полное имя файла,
    System.IO.StreamWriter file = new System.IO.StreamWriter(name);
    file.WriteLine(dataGridView1.RowCount.ToString()); // запись строки,
    file.WriteLine(dataGridView1.ColumnCount.ToString());
    for (int i = 0; i < dataGridView1.RowCount; i++)
      for (int j = 0; j < dataGridView1.ColumnCount; j++)
        file.WriteLine(dataGridView1.Rows[i].Cells[j].Value.ToString());
    file.Close();
    // модальное окно7:
    MessageBox.Show("Данные таблицы записаны в файл.");
  }
  else
    MessageBox.Show("Запись данных таблицы в файл не удалась.");
}

private void button6_Click(object sender, EventArgs e)
{
  // [Л] button6 “Чтение”.
  if (openFileDialog1.ShowDialog() == DialogResult.OK)
  {
    string name = openFileDialog1.FileName;
    System.IO.StreamReader file = new System.IO.StreamReader(name);
    dataGridView1.RowCount = int.Parse(file.ReadLine()); // чтение строки.
    textBox4.Text = dataGridView1.RowCount.ToString();
    dataGridView1.ColumnCount = int.Parse(file.ReadLine());
    textBox5.Text = dataGridView1.ColumnCount.ToString();
    for (int i = 0; i < dataGridView1.RowCount; i++)
      for (int j = 0; j < dataGridView1.ColumnCount; j++)
        dataGridView1.Rows[i].Cells[j].Value = file.ReadLine();
    file.Close();
  }
  else MessageBox.Show("Данные прочитать не удалось.");
}
```

⁷ Модальное окно предназначено для вывода краткого сообщения, выполнение приложения продолжится после его закрытия.

2.2.6. Windows forms. Визуализация данных

Элемент Chart

Одним из средств визуализации данных в приложениях windows form является элемент Chart, используемый для создания диаграмм.

Диаграмма – это графическая форма зависимости между двумя наборами данных. Примерами диаграмм являются графики функции, круговая диаграмма и т.д.

Построение диаграмм осуществляется в свойстве Series элемента Chart. У одного элемента Chart может быть несколько свойств Series.

Следует отметить, что при построении диаграмм *осуществляется масштабирование по обеим осям.*

Основные этапы создания диаграмм

- 1) В окне Поиск по панели элементов набрать chart
- 2) Нарисовать элемент Chart на форме (со свойством Series1 по умолчанию).
- 3) Изменение свойств диаграммы:

[Л] Chart – В окне Свойства: [Л] Series - [Л] 

- добавление/удаление Series:
в окне Члены: [Л] Seriesk - [Л] Добавить или [Л] Удалить
- выбор вида диаграммы:
в окне Свойства Seriesk: [Л]  - [Л] ChartType – [Л] 
Point – для точечного графика, Line – для графика,
- выбор цвета точек (линий, ...) диаграммы:
[Л] Color – [Л]  - выбрать цвет,
- изменение текста легенды:
[Л] LegendText - [Л]  текст.

Основные методы Chart.Series

```
chart1.Series[0].Points.Clear(); // очистка диаграммы Series[0].
```

```
// построение точки:
```

```
chart1.Series[0].Points.AddXY(Math::Round(x, 3), sin(x));
```

2.2.7. Windows forms. Элемент Timer (таймер)

Элемент Timer генерирует событие Tick через каждые Interval мс.

Основные свойства

Interval – интервал между событиями в мс,

Enabled – включает(true)/выключает(false) таймер.

Основное событие

Tick – наступает каждые Interval мс.

Лабораторная работа № 2.6. Windows forms. Визуализация данных

Задание. Добавьте к приложению windows forms, созданному при выполнении ЛБ 2.4 элементы для построения графика функции, заданной в полярной системе координат, $r = f(\varphi)$ Вашего варианта; точки графика должны появляться на графике через 0,1 с.

Требование к проекту

1. Добавить в проект элементы:
пункт меню ЛБ 2.6 и groupBox4;
groupBox4.Text = "название функции Вашего варианта";
groupBox4.Visible = false
2. В элемент groupBox4 добавить элементы:
 - chart1.Series[0] – для построения графика функции,
chart1.Series[0].Visible = false
chart1.Series.ChartType = Line
 - button7 – для начала построения графика,
button7.Text = "Нарисовать"
 - timer1 – для построения точек через 0,1 с.
timer1.Enabled = False
timer1.Interval = 100
3. Объявить глобальную переменную double **phi** для использования в теле обработчика события timer1.Tick.

Содержание отчета

1. Формулировка задания Вашего варианта.
2. Теоретическая часть
преобразование функции Вашего варианта, заданной в полярной системе координат, в прямоугольную систему координат.
3. Расчетная часть
 - добавленные в проект элементы;
 - обоснование выбора элементов и измененные их свойства;
 - объявления глобальной переменной и обработчиков событий с необходимыми комментариями;
 - результаты тестирования.
4. Заключение.
5. Список использованной Вами литературы.

Варианты заданий

1. $r = a \cdot \cos(n \cdot \varphi / 2)$ // n -лепестковая роза.
2. $r = a + b \cdot \varphi$ // архимедова спираль.
3. $r = a \cdot (1 + \cos \varphi)$ // кардиоида.
4. $r^2 = a^2 \cdot \cos 2\varphi$ // лемниската Бернулли.
5. $r^4 - 2a^2 r^2 \cos 2\varphi + a^4 - b^4 = 0$ // овал Кассини.
6. $r = 2 + \cos 3\varphi \sin 2\varphi$ // клевер.
7. $r = 2 - 2 \sin \varphi + \sin \varphi \sqrt{\frac{|\cos \varphi|}{\sin \varphi + 1,4}}$ // сердце.

8. $r = e^{\cos \varphi} 2 - 2 \cos(4\varphi) + \sin^5 \frac{\varphi}{12}$ // бабочка.
9. $r = a + b \cdot \cos(7 \cdot \varphi)$ // семи-лучевая звезда.
10. $r = \left| \cos \frac{\varphi}{4} \right|^3 + \left| \sin \frac{\varphi}{4} \right|^3$ // астроида.
11. $r = 2 + \sin 7\varphi \cos 5\varphi$ // фигуры Лиссажу.
12. $r = 1 + a \cdot \ln(b\varphi) \cdot \cos(c\varphi)$ // волны.
13. $r = 1 + 0.1 \cdot \cos(6\varphi) + 0.05 \cdot \cos(12\varphi)$ // снежинка.
14. $r = 1 + 0.2 \cdot \sin(8\varphi) + 0.1 \cdot \sin(16\varphi)$ // кружево.
15. $r = 2 + \cos(5\varphi)$ // круг с пятью вырезами.

Рекомендации по программированию

Вариант 0. $r = 2 + \cos(3\varphi)$ // круг с тремя вырезами.

Состав проекта представлен на рис. 10.

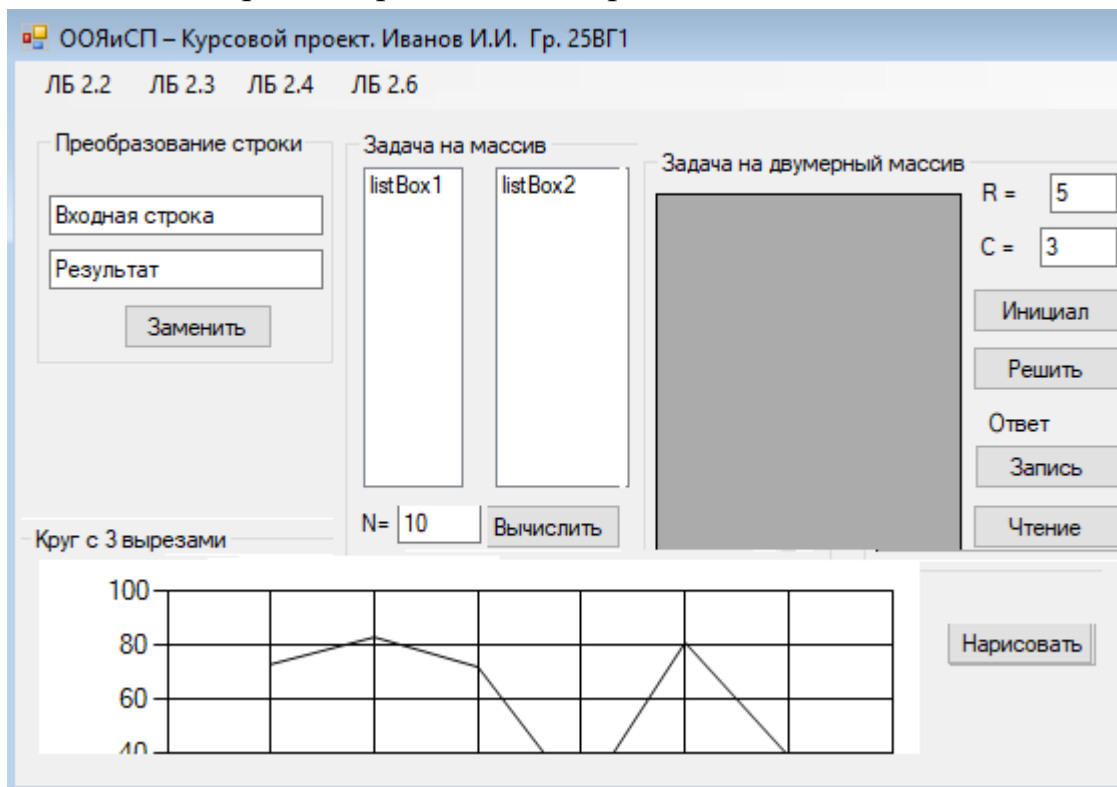


Рисунок 10

Обработчики событий

```
private void лБ26ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox4.Visible = true;
    chart1.Legends.Clear(); // удалили легенду из chart1.
}
```

```

double phi;
private void button7_Click(object sender, EventArgs e)
{
    // [Л] button7 "Нарисовать"
    chart1.Visible = true;
    chart1.Series[0].Points.Clear(); // очистка диаграммы,
    timer1.Enabled = true; // включили таймер,
    phi = 0.0;
    chart1.Height = chart1.Width; // вид chart1 – квадрат,
}
private void timer1_Tick(object sender, EventArgs e)
{
    // прошло 0,1 с = 100 мс
    phi += Math.PI / 100;
    if (phi > 2 * Math.PI) // угол больше 2π, завершаем построений.
    {
        timer1.Enabled = false; // выключили таймер,
        return;
    }
    double r = 2 + Math.Cos(3 * phi),
    // преобразование функции из полярной СС в прямоугольную:
    x = r * Math.Cos(phi),
    y = r * Math.Sin(phi);
    chart1.Series[0].Points.AddXY(1.4*x, 0.7*y); // для симметрии изображения.
}

```

Лабораторная работа 2.7. Windows forms. Моделирование случайных процессов

Задание. Добавьте к приложению windows forms, созданному при выполнении ЛБ 2.6, элементы для моделирования по описанию предметной области случайного блуждания частицы по отрезку с отражающими экранами.

Описание предметной области

- 1) Частица движется по точкам с целочисленными координатами отрезка [0; 77].
- 2) В начальный момент времени частица находится в середине отрезка.
- 3) Каждую десятую долю секунды под воздействием случайных сил частица перемещается на 1 с вероятностью p влево, а с вероятностью $1 - p$ - вправо.
- 4) Если частица находится в левом (в правом) конце отрезка, и она должна переместиться влево (вправо), то частица смещается на 1 вправо (влево), то есть отражается от экрана.

Требование к проекту

1. Добавить в проект элементы:
 - groupBox5 – контейнер для добавляемых элементов;
 groupBox.Text = "Случайное блуждание с отражающими экранами"
 - dataGridView2 – для моделирования отрезка;
 - button8.Text = "Старт" – начало/продолжение блуждания частицы,
 - button9.Text = "Стоп" – останов блуждания частицы,
 - timer2 – для моделирования блуждания,
 timer2.Enabled = False
 timer2.Interval = 100
 - пункт меню "ЛБ 2.7", выбор которого делает видимым groupBox5.
2. Добавить в проект объявление глобальной переменной int nC для запоминания координаты частицы между событиями timer2_Tick.

Содержание отчета

1. Формулировка задания.
2. Расчетная часть
 - добавленные в проект элементы;
 - обоснование выбора элементов и измененные их свойства;
 - объявления глобальной переменной и обработчиков событий с необходимыми комментариями;
 - результаты тестирования.
3. Заключение.
4. Список использованной Вами литературы.

Рекомендации по программированию

Состав проекта представлен на рис. 11.

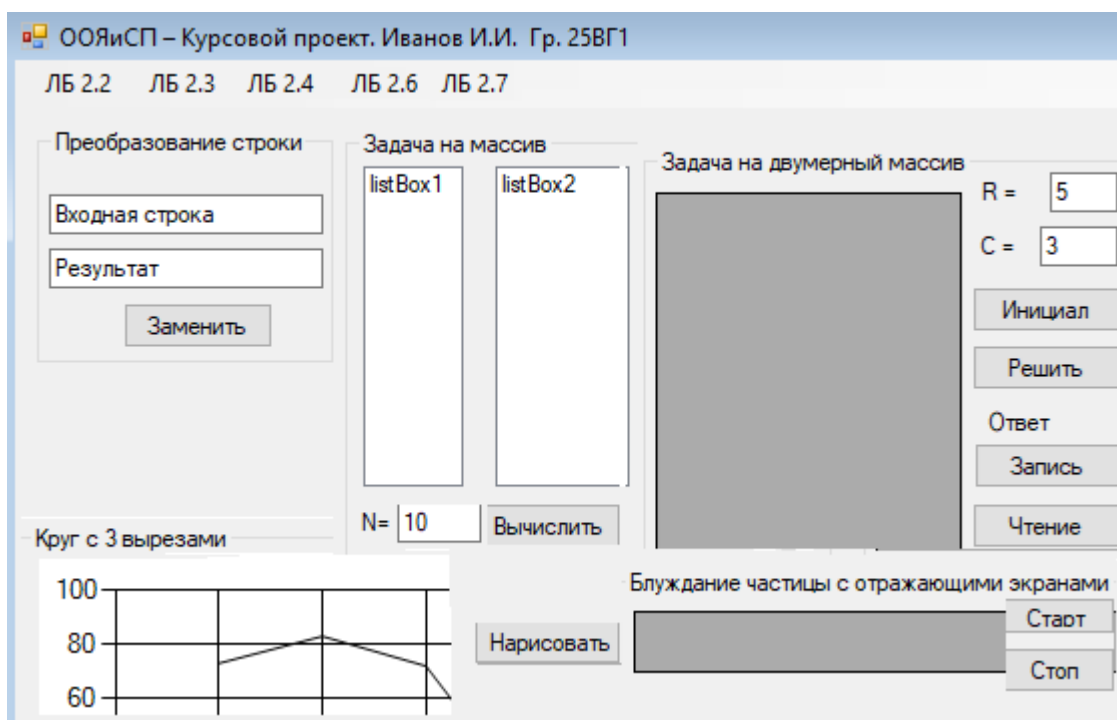


Рисунок 11

Обработчики событий

```
int nC; // глобальная переменная для запоминания координаты частицы.
Random rnd; // для генерации случайных чисел.

private void лБ27ToolStripMenuItem_Click(object sender, EventArgs e)
{
    groupBox5.Visible = true;
    // Изменяем свойства dataGridView2:
    dataGridView2.ColumnCount = 78;
    dataGridView2.RowCount = 1;
    // Скрываем заголовки:
    dataGridView2.ColumnHeadersVisible = false; // столбцов,
    dataGridView2.RowHeadersVisible = false; // строк,
    for (int c = 0; c < dataGridView2.ColumnCount; c++)
        dataGridView2.Columns[c].Width = 7; // ширина столбца = 7,
    // начальное положение частицы:
    nC = dataGridView2.ColumnCount / 2; // координата точки,
    // цвет ячейки:
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.Red;
    rnd = new Random(12345); // инициализация генератора случайных чисел.
}

private void button8_Click(object sender, EventArgs e)
{
    // [Л] button8 "Старт"
    timer2.Enabled = true; // включили timer2
}

private void button9_Click(object sender, EventArgs e)
{
    // [Л] button8 "Стоп"
    timer2.Enabled = false; // выключили timer2.
}

private void timer2_Tick(object sender, EventArgs e)
{
    // Прошла 0,1 с., моделируем смещение частицы:
    double p = 0.4; // вероятность смещения точки влево,
    double t = rnd.NextDouble(); // получили очередное случайное число,
    if (t <= p && nC > 0)
    {
        // выпало смещение влево и частица не у левого конца отрезка:
        // в ячейке, где частица меняем цвет на белый:
        dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.White;
        --nC; // смещение влево,
        // в ячейке, в которую сместилась частица меняем цвет на красный:
        dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.Red;
    }
}
```

```

if (t <= p && nC == 0)
{
    // выпало смещение влево и частица у левого конца отрезка:
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.White;
    ++nC; // отражение частицы от левого экрана вправо,
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.Red;
}
if (t > p && nC < dataGridView2.ColumnCount - 1)
{
    // выпало смещение вправо и частица не у правого конца отрезка:
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.White;
    ++nC; // смещение вправо,
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.Red;
}
if (t > p && nC == dataGridView2.ColumnCount - 1)
{
    // выпало смещение вправо и частица у правого конца отрезка:
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.White;
    --nC; // отражение частицы от правого экрана влево.
    dataGridView2.Rows[0].Cells[nC].Style.BackColor = Color.Red;
}
}
}

```

Результаты тестирования созданного приложения windows forms представлены на рис. 12.

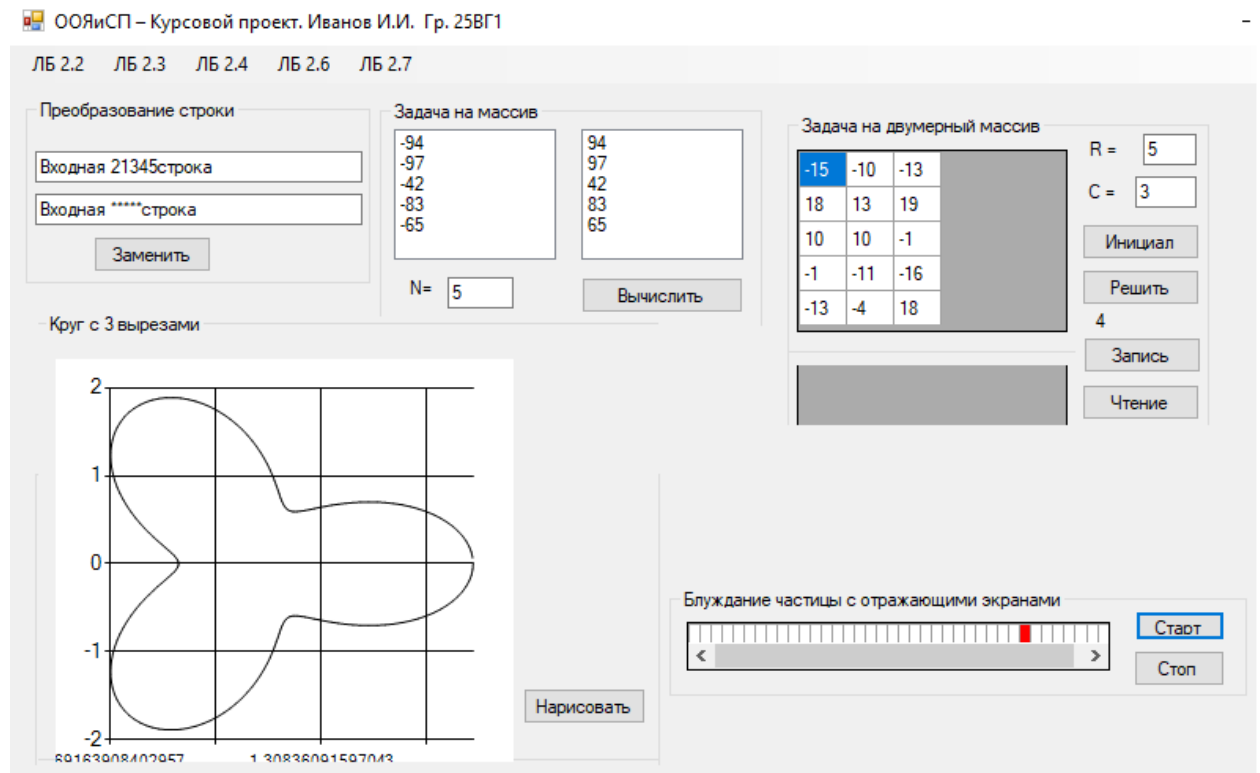


Рисунок 12

2.5 Алгоритмы (C++)

В библиотеке стандартных шаблонов STL объявлено большое количество шаблонов функций от аргументов контейнеров и других последовательностей. Эти шаблоны функций называют *шаблонами алгоритмов* или, кратко, *алгоритмами*⁸.

Объявления алгоритмов находится в заголовочном файле <algorithm>, при этом необходимо использовать пространство имён std.

Следует отметить, что практически все алгоритмы, объявленные в STL, достаточно просто объявить самому, поэтому ограничимся рассмотрением некоторых алгоритмов на примере контейнера vector.

```
#include <vector>
#include <algorithm>
using namespace std
// Объявление вектора v элементов типа T:
vector <T> v(20);
// Инициализация вектора v: ...
// Объявление итератора:
vector <T>::iterator it;
```

Алгоритм count

```
int count(v.begin(), v.end(), T el)
    возвращает количество вхождений элемента el в вектор
```

Алгоритм find

```
it = find(v.begin(), v.end(), T el)
    если элемент el есть в векторе, то возвращается итератор на первое его
    вхождение в вектор, иначе возвращается итератор на конец вектора.
```

Пример.

```
it = find(v.begin(), v.end(), 45);
if(it != v.end()) cout<<"\n Элемент "<< *it<< "принадлежит вектору";
else cout<<"\n Элемент 45 не принадлежит вектору";
```

Алгоритм for_each

вызывает для каждого элемента последовательности заданную функцию.

Пример.

```
// объявление функции:
void show_SQR (int el) {cout <<"\n "<<el*el;}
// вызов функции для каждого элемента последовательности:
for_each(v.begin(), v.end(), show_SQR);
```

Алгоритм reverse

«переворачивает» последовательность.

Пример.

```
reverse(v.begin(), v.end());
```

⁸ Достаточно подробно алгоритмы рассмотрены в учебнике Т.А. Павловская «C/C++. Программирование на языке высокого уровня. – СПб.: Питер, 2010. См. также <http://www.cplusplus.com/reference/algorithm/>

Алгоритм sort

сортирует последовательность по **не убыванию** за время $O(N \ln N)$, где N количество элементов в последовательности.

Пример.

```
it = v.begin();  
sort(it+1, v.end()); // сортировка элементов с 2-го по последний.
```

Алгоритм next_permutation

осуществляет очередную перестановку последовательности в лексикографическом порядке и возвращает true, если следующая перестановка существует, иначе – false.

Пример. Получение всех перестановок вектора v.

```
// Сортировка вектора по не убыванию:  
sort(v.begin(), v.end()); // начальное расположение элементов вектора,  
// получаем все v.size()! перестановки:  
while(next_permutation(v.begin(), v.end()))  
    // вывод очередной перестановки вектора в консоль
```

Алгоритмы min_element и max_element

min_element возвращает итератор на элемент последовательности с наименьшим значением, а max_element – с наибольшим.

Пример.

```
it = min_element(v.begin(), v.end());  
cout<< "\n Наименьший элемент "<< (*it);  
it = max_element(v.begin(), v.end());  
cout<< "\n Наибольший элемент "<< (*it);
```

Глава 3. Объектно-ориентированный анализ.

Статические UML-диаграммы

Объектно-ориентированный анализ (ООА) – это этап разработки программного обеспечения, на котором, во-первых, изучается предметная область (проблема) и, во-вторых, эту область представляют в виде системы взаимодействующих объектов, не задумываясь о технической реализации (языке программирования, базах данных и т.д.).

Целью ООА является понимание и формализация требований к системе, выделение ключевых сущностей, их свойств и взаимодействий.

Ключевыми задачами ООА являются:

- ✓ выявление классов и сущностей (объектов): кто главные «действующие лица» в системе? Например, Пользователь, Заказ, Товар, Корзина.
- ✓ определение свойств (атрибутов) сущностей: какими данными обладает каждая сущность? У Заказа, например, есть номер, дата, статус.
- ✓ определение методов (поведения): что каждая сущность может делать? Например, Заказ может рассчитатьСумму(), изменитьСтатус().
- ✓ определение взаимосвязей: как сущности связаны друг с другом? возможные виды связей: наследование, ассоциация, агрегация, композиция.

- ✓ выявление сценариев использования: как пользователи и другие системы будут взаимодействовать с нашей системой для достижения целей?

Результатом ООА является модель предметной области, которая точно описывает, ЧТО должна делать система, а не КАК. Для визуализации этой модели чаще всего и используются статические UML-диаграммы.

3.1. Статические UML-диаграммы объектной области.

Диаграммы классов

UML (Unified Modeling Language) — это стандартизированный графический язык для визуализации, специфицирования, конструирования и документирования артефактов программных систем.

Статические диаграммы показывают неизменную (статическую) структуру системы. Они отвечают на вопрос «Из каких частей состоит система?» и не показывают ее поведение во времени.

Основной и наиболее часто используемой статической диаграммой является *диаграмма классов*.

Диаграмма классов отражает структуру системы в терминах классов, их свойств, методов и взаимоотношений между ними.

Создание диаграммы классов в MS Visio 2002

- Открыть MS Visio.
- File – New – Database – Database Model Diagram
- Добавление классов и их свойств (атрибутов):
 - перетащите фигуру Класс (Class) на холст;
 - [Л]² Класс;
 - в окне Class Properties:
 - [Л] Class – задание имени класса;
 - [Л] Attributes – задание имени, типа, видимости и начального значения свойства;
 - [Л] Operations – задание имени, параметров, типа значения, видимости и тела метода;
Polymorphic – виртуальный метод,
тело метода не отображается на диаграмме классов;
 - создание наследования классов:
соединить стрелкой Generalization дочерний класс с родительским;
 - спецификаторы доступа: public (+), private (-), protected (#);
 - Ar1: float [] – одномерный массив элементов типа float;
 - Ar2: T[][] – двумерный массив элементов типа T;
 - имя_класса <T> - шаблон класса.

Примечание. 1. В диаграмме классов не отображаются тела классов по следующим причинам:

- диаграмма классов должна быть читаемой и показывать архитектуру, а не детали реализации (абстракция);

- диаграмма классов создаётся для проектировщиков, код – для программистов (разделение ответственности);
 - диаграмма классов показывается только то, что нужно для понимания взаимодействия между классами (сокрытие информации).
2. Если нужно показать тела методов (реализацию), то следует использовать псевдокод в документации.

Список литературы

1. Довбуш Г. Ф. Visual C++ на примерах / Г. Ф. Довбуш , А. Д. Хомоненко; под ред. А. Д. Хомоненко. — СПб.: БХВ-Петербург, 2008.
2. Иванова Г.С. Технология программирования: учебник. – М.: Изд-во МГТУ им. Баумана, 2020.
3. Корера А., Фрейзер С., Джентайл С. и др. Visual C++ .Net: Пособие для разработчиков. – М.: Лори, 2003 – 398 с.
4. Павловская Т.А. C/C++. Программирование на языке высокого уровня: Учебник для бакалавров и магистров. – СПб.: Питер, 2012.
5. Поляков А. Б., Шестаков А. Л. Объектно-ориентированное программирование на C++. – М. : Изд-во МГТУ им. Н. Э. Баумана, 2021. – 446 с.
6. Прата С. Язык программирования C++. Лекции и упражнения. — СПб.: Диалектика, 2022. — 1248 с.
7. Рендольф Н., Гарднер Д., Минутилло М. и др. Visual Studio для профессионалов. – М.: ООО «И.Д. Вильямс, 2011. – 1184.
8. Страуструп Б. Программирование: принципы и практика с использованием C++. — 2-е изд. — СПб.: Диалектика, 2020. — 1242 с.
9. Троелсен Э., Джепикс Ф. Язык программирования C# 10 и платформа .NET 6. — СПб.: Диалектика, 2023. — 1088 с.
10. Фридман А.Л. Объектно-ориентированное программирование на языке СИ++ . - М. : Горячая Линия - Телеком, 2001.
11. Фримен Э., Робсон Э. Паттерны проектирования. — 2-е изд. — СПб.: Питер, 2021. — 656 с.
12. Шибанов С.В. Основы программирования на языке C++: учебное пособие. – Пенза: Изд-во Пенз. гос. ун-та, 2008.

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**

Кафедра «Компьютерные технологии»

Направление подготовки **01.03.02 Прикладная математика и информатика**

Классы, экземпляры классов

(Наименование выполняемой работы)

**по дисциплине «Объектно-ориентированные языки и системы
программирования»**

ОТЧЕТ

по лабораторной работе № XX, вариант № XX

(Обозначение документа)

Студент
гр. 2ХВГ1

(Подпись)

Петров П.П.
(Фамилия И.О.)

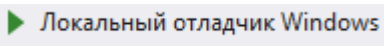
Преподаватель
к.ф.-м.н, доцент

(Подпись)

Горюнов Ю.Ю.
(Фамилия И.О.)

Пенза, 202Х

Приложение Б. Создание консольных проектов в MS Visual Studio 2019/2022

- 1) Открыть MS Visual Studio.
- 2) [Л] Создание проекта
Все языки: [Л] C++
Все платформы: [Л] Windows
Все типы проектов: [Л] Консоль
[Л] Консольное приложение
[Л] Далее
- 3) Имя проекта: набрать
Расположение: выбрать
[Л] Создать
- 4) Открыть консольное приложения: [Л] 
- 5) **Изменение цветов** шрифта и фона в окне открытого консольного приложения:
[П]  - [Л] Свойства – [Л] Цвета
Текст на экране - выбрать
Фон экрана - выбрать

Пример консольного приложения

```
#include <iostream>
#include <Windows.h> // для SetConsoleCP и SetConsoleOutputCP.
using namespace std;
int main()
{SetConsoleCP(1251); // установка кодировки 1251 для ввода в консоль,
 SetConsoleOutputCP(1251); // установка кодировки для вывода в консоль,
  cout << "Привет";
  cout << "\n\n"; system("pause"); return 0;
}
```

Приложение В. Создание приложений windows forms на языке C# в MS VS 2019/2022

1. Открыть MS VS 2019/2022
2. [Л] Создание проекта
3. В поиске шаблонов: Windows Forms
4. [Л] Windows Forms (.NET Framework) на C#
5. [Л] Далее
6. Имя проекта: WinFormsC#
7. Выбрать расположение проекта
8. [Л] Создать

Вставка элементов в приложение windows forms

- 1) открыть окно имя_формы.h [Конструктор]: [Л]² MyForm.h (имя формы)
- 2) открыть панели элементов и их свойств:
Ctrl + ALT + X; на Панели элементов: [Л] ▾ - [Л] Закрепить
Alt + Enter; на Панели свойств: [Л] ▾ - [Л] Закрепить
- 3) нарисовать элемент на форме:
[Л] по элементу на Панели элементов – [Л] на форме.
- 4) Перетащить элемент на нужное место на форме и изменить его размеры.
- 5) Изменить свойства элемента:
 - а) выделить элемент на форме,
 - б) в окне «Свойства» (рис. 13) в левом столбце выбрать свойство, а в правом ввести новое значение.

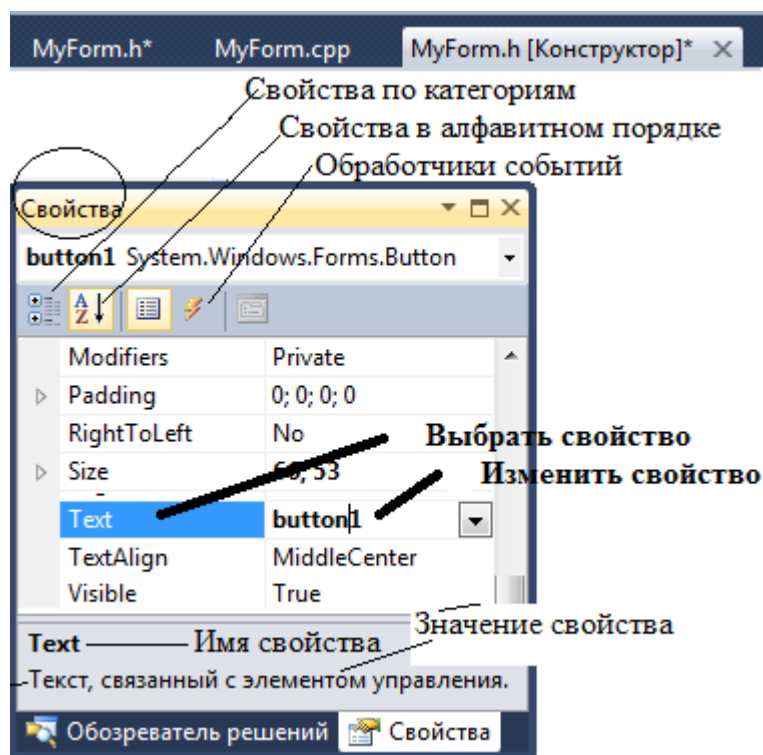


Рисунок 13

- 6) Объявить обработчики событий для элемента:
 - а) выбрать на форме элемент,

б) в окне «Свойства» (рис. 14), открыть закладку «События» ,

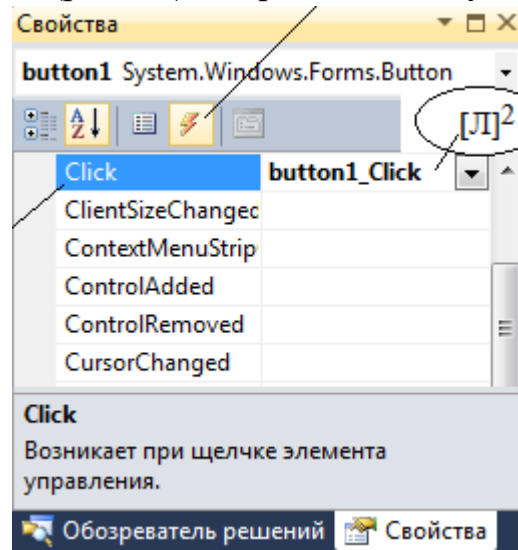


Рисунок 14

в) в левом столбце выбрать событие, [Л]² в правом столбце откроет «заготовку» обработчика события в окне кода, например,

```
private: System::Void button1_Click(System::Object^ sender,
System::EventArgs^ e) {
    операторы – тело обработчика события с именем button1_Click
}
```

Примечание. `button1` – это имя элемента, для которого объявляется обработчик события, а `Click` – название события (щелчок левой клавишей мыши по элементу).

Открытие приложения windows forms

Отладка – Начать отладку, или F5 или 

Примечание. Если в проекте не обнаружено ошибок, то будет создан exe-файл по адресу *папка_с_проектом\bin\debug\имя решения.exe*, который будет выполняться только на компьютерах с ОС Windows и установленной MS VS Studio.

Приложение Г. Открытие консольных приложений в MS VS 2019/2022, созданных в MS VS 2022/2019

- Для открытия в MS VS 2022 приложения, созданного в MS VS 2019, следует в файле `.vsxproj` в Текстовом редакторе, используя Поиск, заменить 143 на 142 во всех строках:

`<PlatformToolset>v143</PlatformToolset>`,

после сохранения файла в проекте: [Л] Обновить все

- Для открытия в MS VS 2019 приложения, созданного в MS VS 2022, следует в файле `.vsxproj` в Текстовом редакторе, используя Поиск, заменить 142 на 143 во всех строках:

`<PlatformToolset>v142</PlatformToolset>`,

после сохранения файла в проекте: [Л] Обновить все

Содержание

Введение. Объектно-ориентированные языки и системы программирования	3
Глава 1. Объектно-ориентированные языки программирования	3
1.1. Стили программирования, основные принципы, история развития	3
1.2. Тип класс, экземпляры класса	4
1.3. Конструкторы и деструкторы классов	6
Лабораторная работа № 1.1. Классы, экземпляры классов	7
Рекомендации по выполнению задания	8
1.4. Наследование классов	10
1.4.1. Полиморфизм и виртуальные методы	11
Лабораторная работа № 1.2. Конструкторы, деструкторы, наследование, полиморфизм	12
Рекомендации по выполнению задания	14
1.5. Шаблоны классов	16
Лабораторная работа № 1.3. Шаблоны классов	17
Рекомендации по программированию	17
1.6. Исключения и их обработка	19
1.7. Перегрузка операций	21
Лабораторная работа № 1.4. Перегрузка операций. Обработка исключения	23
1.8. Поточные классы	24
1.9. Файловые потоки	26
Лабораторная работа № 1.5. Файловые операции	28
Рекомендации по программированию	28
1.10. Строковые потоки	28
Лабораторная работа № 1.6. Использование строкового типа	29
1.10. Контейнерные классы	30
1.10.1 Последовательный контейнер vector (вектор)	31
1.10.2 Ассоциативный контейнер map (словарь)	33

Глава 2. Объектно-ориентированные системы программирования	35
2.1. Создание приложений windows forms в MS VS 2019/2022	35
2.2. Основные элементы для создания приложений windows forms .	37
2.2.1 Форма - - контейнер для элементов	37
2.2.2 MenuStrip – меню	38
Лабораторная работа № 2.1. Windows forms. Элементы форма и меню	38
Рекомендации по выполнению задания.....	39
2.2.3 Элементы Метка, Кнопка, Редактор, Рамка с надписью	39
Button (.....	39
Лабораторная работа № 2.2. Windows forms. Элементы «кнопка», «редактор».....	39
Рекомендации по программированию	41
2.2.4. Элемент Список	42
Лабораторная работа № 2.3. Windows forms. Элемент Список. Контейнер вектор	43
Рекомендации по программированию	44
2.2.5. Элемент Таблица Данных	45
Лабораторная работа № 2.4. Windows forms. Элемент Таблица Данных	46
Рекомендации по программированию	47
Лабораторная работа 2.5. Windows forms. Элементы OpenFileDialog и SaveFileDialog	49
Рекомендации по программированию	49
2.2.6. Windows forms. Визуализация данных Элемент Chart.....	51
2.2.7. Windows forms. Элемент Timer (таймер).....	51
Лабораторная работа № 2.6. Windows forms. Визуализация данных	52
Рекомендации по программированию	53
Лабораторная работа 2.7. Windows forms. Моделирование случайных процессов	54
Рекомендации по программированию	55
2.5 Алгоритмы (C++)	58
Глава 3. Объектно-ориентированный анализ. Статические UML-диаграммы.....	59
3.1. Статические UML-диаграммы объектной области. Диаграммы	

классов.....	60
Список литературы	62
Приложение А. Титульный лист отчёта о ЛБ	63
Приложение Б. Создание консольных проектов в MS Visual Studio 2019/2022.....	64
Приложение В. Создание приложений windows forms на языке C# в MS VS 2019/2022.....	64
Вставка элементов в приложение windows forms.....	64
Открытие приложения windows forms.....	66
Приложение Г. Открытие консольных приложений в MS VS 2019/2022, созданных в MS VS 2022/2019	66